

A4 — Constrained Shortest Paths

CS 3050: Data Structures

Tianyi Zhang

University of Missouri — Columbia

April 2026

1. Introduction

The shortest-path problem is fundamental in graph algorithms: given a weighted graph and two vertices, find the path of minimum total weight between them. Dijkstra's algorithm solves this efficiently for non-negative edge weights by greedily expanding the cheapest frontier node. Its correctness relies on a critical property: the search state is *memoryless*. The only information needed at a vertex is the cumulative cost to reach it. Path history is irrelevant to the next decision, which is why Dijkstra can lock each vertex on first expansion via a visited array.

Real-world vehicles — aircraft, ground robots, trucks — have kinematic constraints. A vehicle with a finite turning radius cannot make arbitrarily sharp turns. When we arrive at a vertex, the *direction from which we arrived* determines which outgoing edges are physically feasible. The search state is no longer memoryless: it must encode the arrival trajectory.

This creates a fundamental incompatibility with standard Dijkstra. When Dijkstra pops a vertex from the priority queue it is locked as visited under the guarantee that no cheaper path exists. Under a turn constraint, the cheapest arrival at a vertex might come in at an angle that makes turning toward the destination impossible. A more expensive arrival along a different edge might be the only kinematically feasible route. Locking the vertex on its first expansion would trap the algorithm and miss the true optimal path.

Inserting a local angle check into Dijkstra — rejecting outgoing edges that turn too sharply — does not solve the problem either, for two reasons. First, angle-only checks are defeated by the *discretization trap*: in a high-resolution grid, very short edges can chain together to approximate a physically impossible turn through a sequence of individually legal micro-maneuvers. Second, the visited array still prevents alternative trajectories through previously visited vertices.

This assignment implements and tests a turn-constrained Dijkstra algorithm with four key modifications: (1) an expanded state space where the search key is the pair **(vertex, incoming edge)** rather than just vertex, restoring the memoryless property and allowing the textbook Dijkstra proof to apply; (2) removal of the visited array in favour of a parent-tree (history array) that permits vertex revisits with different incoming edges; (3) a minimum edge length requirement of $L_{\min}/2$ to bound realised curvature and prevent micro-maneuvers; and (4) a Priority Search Tree (PST) at each vertex that answers 2D angle-and-length range queries in $O(\log n + k_{\text{valid}})$ time, avoiding the candidate explosion of naive linear filtering.

2. Algorithms Implemented

2.1 Graph Representation

The graph is a geometric adjacency list. Vertices are placed on a regular grid over a rectangular environment with configurable resolution (cells per meter). Each vertex stores integer grid coordinates (x, y) and a unique ID computed as $\text{row} \times \text{cols} + \text{col}$. Each directed edge stores source and destination IDs, the Euclidean edge weight, and two precomputed angles — the outgoing angle from the source (*angle_leaving_src*) and the incoming angle at the destination (*angle_entering_dst*). Angles are computed via $\text{atan2}(dy, dx)$ and normalised to $[0, 360)$. For every directed edge $u \rightarrow v$, the reverse edge $v \rightarrow u$ is also constructed with appropriately reversed angles.

Obstacles are convex polygons (rectangles in all test cases). During graph construction, any edge whose line segment would intersect an obstacle is excluded. Intersection testing combines segment-vs-polygon-edge checks (including the collinear-overlap case) with multiple sample-point containment checks along each edge to catch boundary-skimming trajectories.

2.2 Standard Dijkstra (Baseline)

The baseline Dijkstra implementation follows the textbook algorithm we've been learning in class. The priority queue holds (vertex, cumulative_cost) pairs. A distance array tracks the best-known cost to each vertex, and a visited array prevents re-expansion. On each pop, if the vertex is unvisited, it is marked visited and all adjacent edges are relaxed. Path reconstruction uses a predecessor array. A binary min-heap serves as the priority queue, giving the standard $O((|V| + |E|) \log |V|)$ running time.

2.3 Turn-Constrained Shortest Path

The constrained algorithm is Dijkstra's algorithm applied to an *expanded state-space graph*. Each state is the pair

$(\text{vertex}, \text{incoming_edge})$

where *incoming_edge* is the directed edge used to arrive at *vertex*. The source vertex is assigned a sentinel incoming-edge ID since it has no arrival direction. In this expanded space, the standard Dijkstra dominance relation $d[v] = \text{min cost to reach } v$ becomes $\text{best_cost}[(v, e)] = \text{min cost to reach } v \text{ via edge } e$. Each state is processed at most once at its final cost, exactly as the textbook proof requires. The algorithm terminates when the destination vertex is popped (in any state) because any alternative path to the destination must be more expensive.

State space via edge IDs. Every directed edge in the graph is assigned a unique integer ID (0 through $|E|-1$) at algorithm start, with $|E|$ reserved as the sentinel source state. A hash map from Edge^* to integer ID provides $O(1)$ lookups during relaxation. The *best_cost* array is one-dimensional of size $|E|+1$, giving exact dominance tracking without any approximation.

Turn validation. When expanding state (u, e_{in}) , the arrival trajectory is known from the edge's *angle_entering_dst*. The *continuation direction* is computed as $(\text{arrival_angle} + 180) \bmod 360$ — the direction the vehicle would continue if going straight. An outgoing edge e_{out} is kinematically valid only if its *angle_leaving_src* falls within $\pm\theta_{\text{max}}$ of the continuation direction. At the source vertex, all outgoing edges are valid regardless of angle because the vehicle has no prior trajectory.

Length enforcement ($L_{\min}/2$ theorem). To prevent micro-maneuvers, every edge traversed must have weight $\geq L_{\min}/2$. This guarantees that every two consecutive edges in a path span at least $L_{\min}$ total distance, bounding the realised curvature to $\kappa_{\max} \approx \theta_{\max}/L_{\min}$. The benefit of the half-length formulation is that the complex pairwise constraint (checking pairs of edges for combined length) reduces to a simple per-edge check.

No visited array. Because the state includes an incoming edge, two distinct arrivals at the same vertex via different edges are different states and are tracked separately. The parent tree (history array of {node_id, parent_index, incoming_edge_id} records) replaces the visited array and enables path reconstruction even in the presence of vertex revisits.

Lazy decrease-key. The min-heap implementation in this codebase does not expose handles, so Q.modifyKey from the lecture pseudocode is implemented via *lazy deletion*: when a relaxation finds a better cost for a state, the updated entry is pushed onto the heap without removing the stale entry. At pop time, if the popped entry's cost exceeds best_cost for its state, it is skipped. This is a standard substitute for handle-based decrease-key that preserves correctness and asymptotic complexity.

2.4 Priority Search Tree (PST)

At each vertex, outgoing edges must be queried by two constraints simultaneously: angle $\in [\theta_{\min} \pm \theta_{\max}]$ and length $\geq L_{\min}/2$. A naive approach — BST on angle, then linearly filter by length — costs $O(\log n + k_{\text{candidates}})$, which degrades badly when many edges satisfy the angle constraint but few satisfy the length constraint.

The Priority Search Tree combines a BST on angle with a max-heap on edge length. Construction: starting from edges sorted by angle, the longest edge becomes the root, remaining edges split at the median angle into left and right subtrees, and the process recurses. Each node stores the maximum edge length within its subtree.

Query: at each node, if the subtree's max length is below $L_{\min}/2$, the entire branch is pruned instantly. Otherwise the node's own edge is tested against both constraints, and recursion descends into left or right based on the angle split. Angular wraparound across $0^\circ / 360^\circ$ is handled by splitting the query into two sub-ranges and unioning the results. When $\theta_{\max} \geq 180^\circ$, the query short-circuits to a full-circle traversal. The query runs in $O(\log n + k_{\text{valid}})$, fully bypassing $k_{\text{candidates}}$.

PSTs are built lazily — only when a vertex is first expanded by the main algorithm — which avoids building trees for vertices that are never reached, such as vertices inside obstacles or in unreachable regions.

3. Testing Plan

Confidence in correctness requires a layered testing approach. No single test can prove correctness in general, but nine distinct tests across four categories provide strong combined evidence. All paths returned by any algorithm are automatically checked for validity — every segment length and every turn angle is verified programmatically — so correctness claims are not dependent on visual inspection alone.

3.1 Designed Scenarios (Tests 1–5)

Five hand-designed scenarios target specific aspects of the algorithm:

Test 1 — Double-Channel Benchmark (provided). Two large obstacles with "trap teeth" form a narrow channel that is mathematically shorter but kinematically invalid. The PST algorithm must reject the short path and find the smooth arc over the top. This is the primary benchmark for correctness.

Test 2 — Open Field (no obstacles). Both algorithms should find the same straight-line optimal path. Confirms the constrained algorithm introduces no unnecessary cost when constraints are non-binding.

Test 3 — U-Turn with tight $\theta_{\max}$. A wall across the middle with a gap at the top forces a detour. Tight $\theta_{\max} = 30^\circ$ forces the PST algorithm to use a smooth multi-edge arc while Dijkstra takes sharp turns. Tests the algorithm under tighter constraints than the benchmark.

Test 4 — Resolution Scaling. Same environment at resolutions 1.0 and 2.0 cells/m. Both algorithms should produce valid paths of identical cost regardless of grid density, confirming that the $L_{\min}/2$ floor correctly prevents resolution-dependent micro-maneuvers.

Test 5 — Zigzag Corridor. Alternating obstacle teeth create a complex multi-obstacle environment. Confirms the algorithm handles non-trivial obstacle fields correctly.

3.2 Structural Component Verification (Test 6)

Test 6 — PST Query Correctness vs Brute Force. The PST data structure is the critical component for efficient 2D range queries. Its correctness is verified directly by comparing its query results against a brute-force linear filter over 12 different query ranges (including wraparound, narrow windows, high length thresholds, and "all edges") at 20 sample vertices — 240 query tests total. Each query must return exactly the same set of edges as brute-force, otherwise the test fails.

3.3 Edge and Negative Cases (Test 8)

Test 8 — Edge Cases. Four boundary cases test algorithm robustness: (a) **src == dst** should return a trivial one-vertex path; (b) **Unreachable destination** (wall blocks all paths with no valid edges crossing) should return NO PATH, not hang or return an incorrect path; (c) **$\theta_{\max} = 0$** (no turns allowed) should only succeed when a perfectly colinear path exists; (d) **$L_{\min}$ larger than all edges** should return NO PATH because no edge qualifies.

3.4 Statistical / Stress Tests (Tests 7, 9)

Test 7 — Randomized Stress Testing. 30 random graphs are generated with random numbers of random obstacles. For each graph, random source and destination vertices are chosen, `shortest_path` is run, and if a path is returned it is validated programmatically. This provides statistical evidence for correctness beyond hand-designed scenarios: if the algorithm has a validity bug that manifests in unusual graph topologies, random testing surfaces it.

Test 9 — Parameter Sweep on $\theta_{\max}$. A fixed scenario is run with $\theta_{\max} \in \{15^\circ, 30^\circ, 45^\circ, 60^\circ, 90^\circ, 180^\circ\}$. This demonstrates smooth behaviour transition: NO PATH at tight constraints, valid paths at larger constraints, with all paths satisfying their respective $\theta_{\max}$.

3.5 Sufficiency

The combination covers: (1) the primary failure mode where Dijkstra finds an invalid shortest path (Tests 1, 3); (2) non-binding-constraint sanity (Tests 2, 4, 5); (3) direct verification of the PST component (Test 6); (4) boundary conditions and degenerate inputs (Test 8); (5) statistical robustness across random topologies (Test 7); and (6) parameter-dependent behaviour (Test 9). Every path in every test is checked against both constraints programmatically, so any validity violation across the ~40 paths evaluated would be caught.

4. Test Results

4.1 Test 1: Double-Channel Benchmark

Parameters: 30×30m environment, resolution 2.0 cells/m (3721 vertices, ~2M directed edges), $\theta_{\max} = 45^\circ$, $L_{\min} = 8.0\text{m}$, max edge distance 12.0m. Start (2.0, 15.0), End (28.0, 15.0). Four rectangular obstacles including two trap teeth.

Standard Dijkstra: path length 27.17 m — INVALID

Edge	Segment	Length (m)	Turn (°)	Valid?
1	1834 → 1974	9.06	N/A	OK
2	1974 → 1922	4.53	12.68	OK
3	1922 → 1800	1.00	83.66	FAIL [SHORT][TURN]
4	1800 → 1747	4.03	82.87	FAIL [TURN]
5	1747 → 1886	8.56	13.83	OK

Dijkstra's path passes through the narrow trap channel. Edge 3 is 1.00m (below 4.0m minimum) with an 83.66° turn (nearly $2\times$ the 45° limit). This is clearly kinematically infeasible.

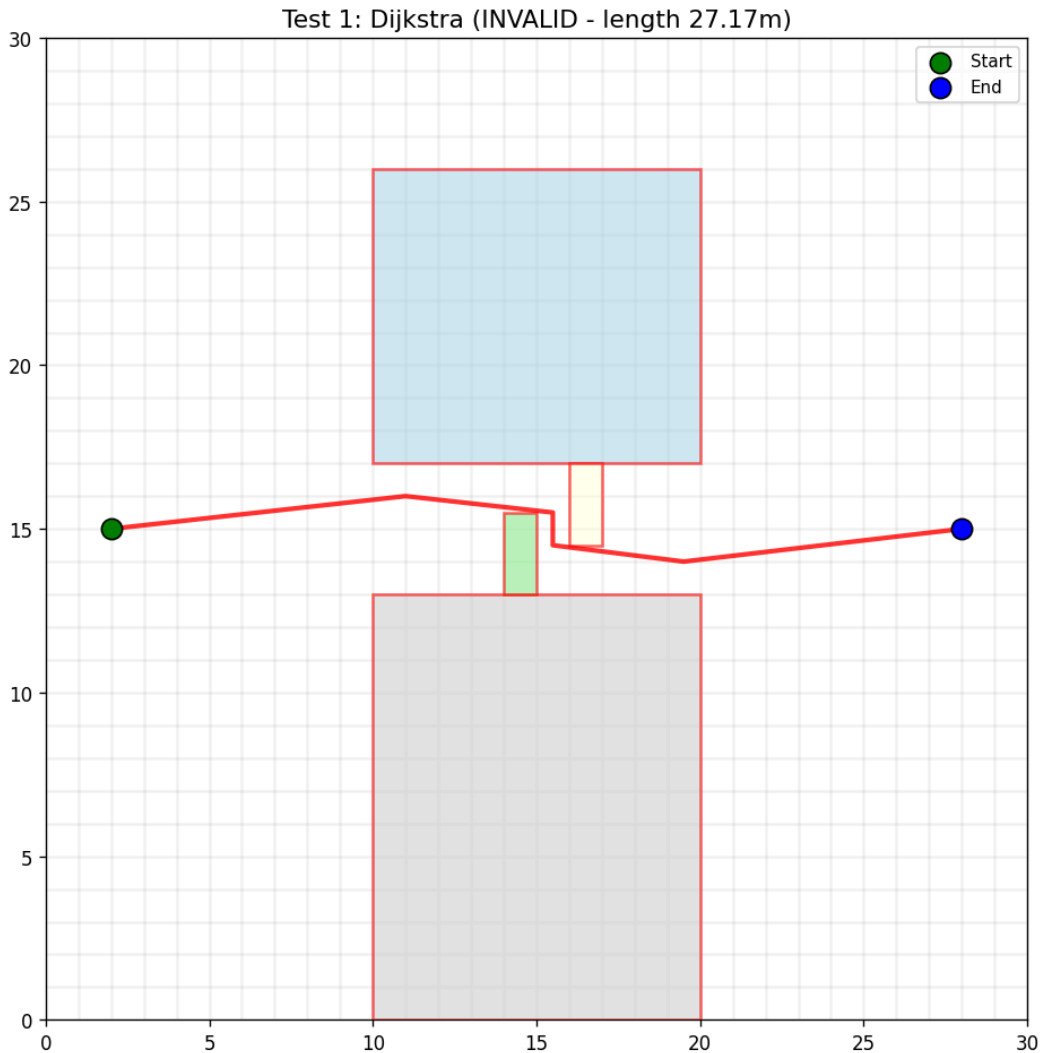


Figure 1. Test 1 — Standard Dijkstra path (27.17m, INVALID). The path threads the narrow trap channel between obstacles.

PST Constrained: path length 37.75 m — VALID (188,799 expansions)

Edge	Segment	Length (m)	Turn (°)	Valid?
1	1834 → 2573	6.95	N/A	OK
2	2573 → 3191	6.40	8.40	OK
3	3191 → 3261	4.53	45.00	OK
4	3261 → 3213	6.52	10.74	OK
5	3213 → 2792	4.61	45.00	OK
6	2792 → 1886	8.75	9.64	OK

The PST-constrained algorithm correctly rejects the narrow channel and routes around the top obstacle. All six segments are $\geq 4.0\text{m}$ (minimum 4.53m) and all turns are $\leq 45^\circ$ (maximum 45.00° , exactly at the limit). The path is 38.9% longer than Dijkstra's invalid shortcut but fully kinematically valid.

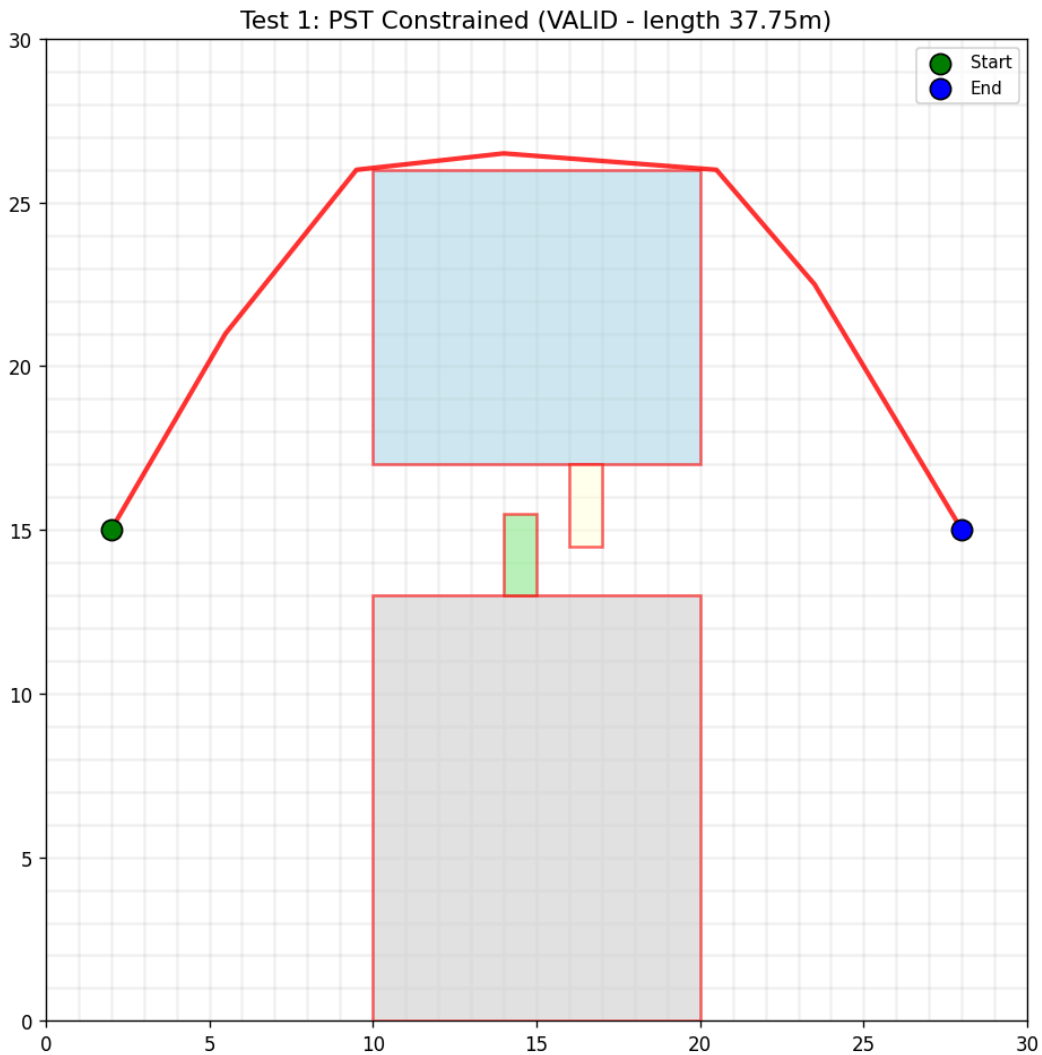


Figure 2. Test 1 — PST Constrained path (37.75m, VALID). The path forms a smooth arc over the top obstacle with all turns $\leq 45^\circ$ and all segments $\geq 4.0\text{m}$.

4.2 Test 2: Open Field (No Obstacles)

Parameters: 20×20m, resolution 1.0, $\theta_{\max} = 45^\circ$, $L_{\min} = 4.0\text{m}$. Start (2.0, 10.0), End (18.0, 10.0).

Algorithm	Path Length	Edges	All Valid?
Standard Dijkstra	16.00 m	2	YES
PST Constrained	16.00 m	2	YES

Both algorithms find the identical 16.00m path (two straight segments). The constrained algorithm introduces no unnecessary cost when a straight-line path is feasible. 14,481 expansions — the constrained algorithm must still explore many states because it cannot lock vertices, but this does not affect solution quality.

4.3 Test 3: U-Turn Scenario

Parameters: 20×20m, resolution 2.0, $\theta_{\max} = 30^\circ$ (tight), $L_{\min} = 6.0\text{m}$. Wall at $x \in [9, 11]$ from $y=0$ to $y=15$ with gap above. Start (5.0, 5.0), End (15.0, 5.0).

Dijkstra: 24.35m — INVALID (sharp turns 51.91° and 53.13° exceed 30° limit)

PST: 32.32m — VALID (10 edges, max turn 29.20° , min length 3.00m)

Edge	Length (m)	Turn ($^\circ$)	Valid?
1	3.20	N/A	OK
2	3.04	29.20	OK
3	3.16	27.90	OK
4	3.54	26.57	OK
5	3.16	26.57	OK
6	3.04	27.90	OK
7	3.20	29.20	OK
8	3.81	28.14	OK
9	3.00	23.20	OK
10	3.16	18.43	OK

Under the tighter 30° constraint, the PST algorithm produces a longer, smoother 10-edge arc where every turn is within 1° of the 30° limit (max 29.20°). Dijkstra's 6-edge path has individual turns of 51.91° and 53.13° , nearly double the limit.

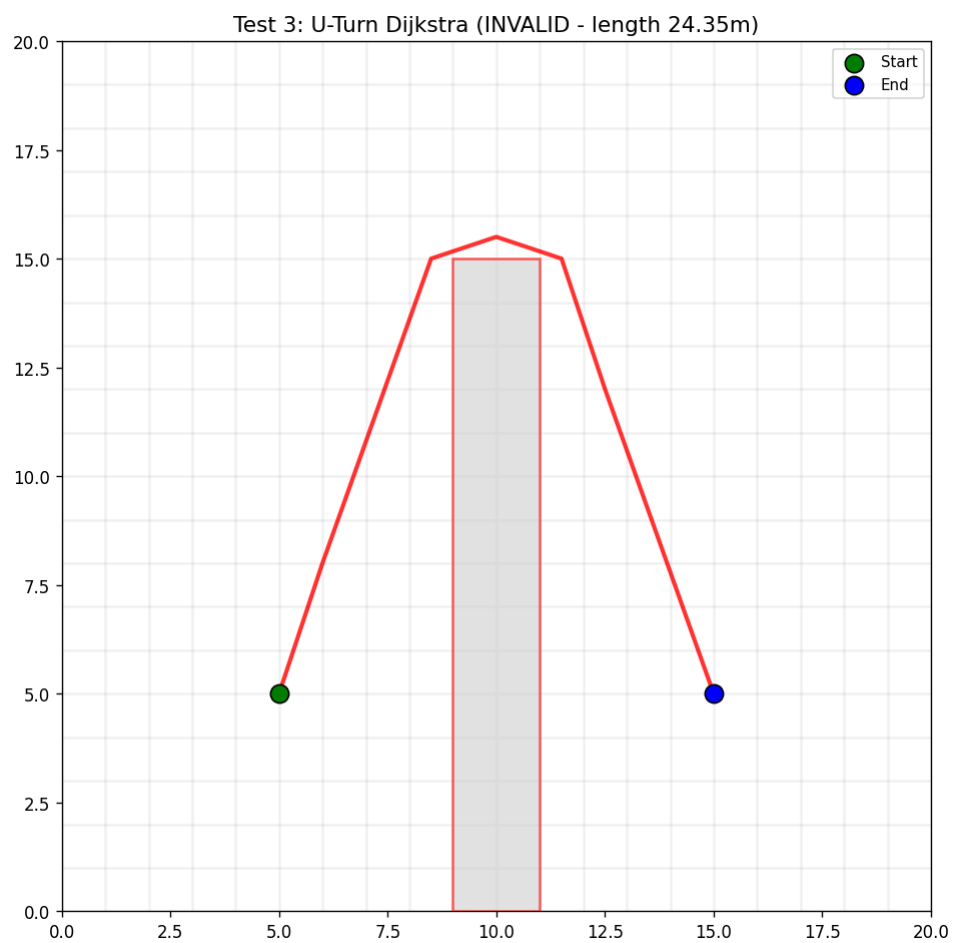


Figure 3. Test 3 — Dijkstra (24.35m, INVALID). Sharp angular transitions at the gap entry/exit violate $\theta_{max} = 30^\circ$.

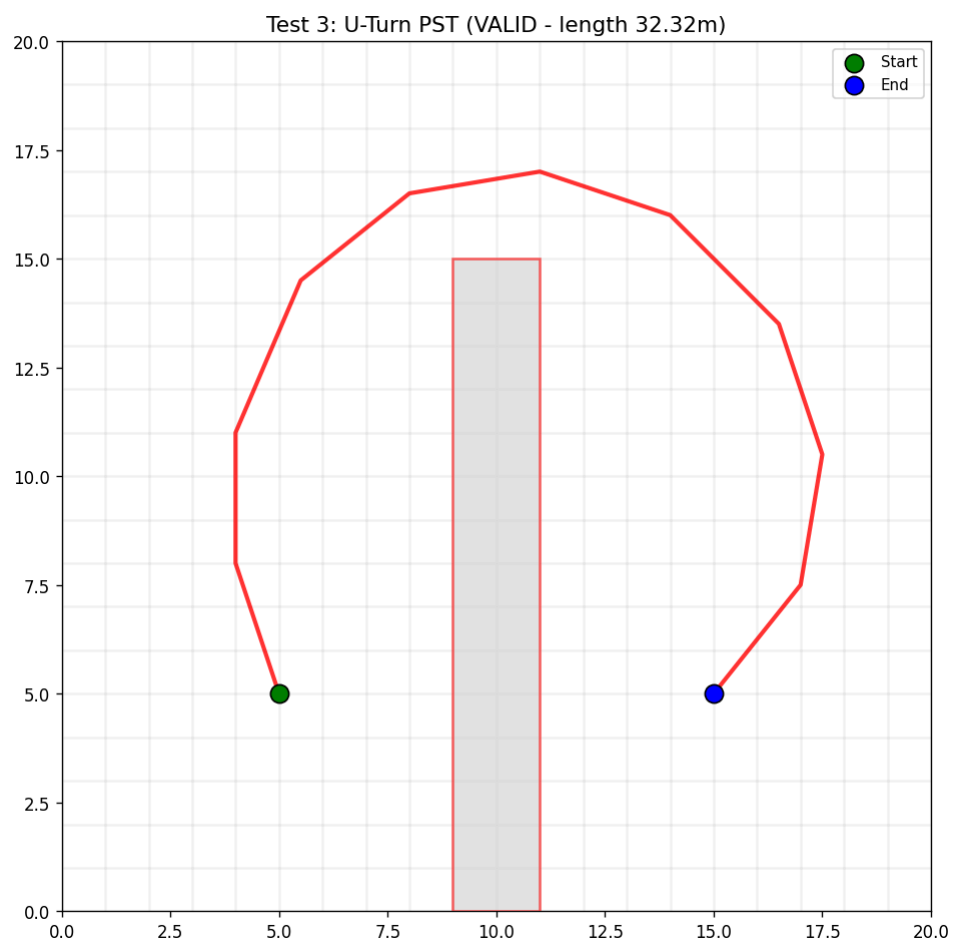


Figure 4. Test 3 — PST Constrained (32.32m, VALID). A smooth 10-segment arc enables navigation under $\theta_{max} = 30^\circ$.

4.4 Test 4: Resolution Scaling

Parameters: 20×20m, $\theta_{\max} = 45^\circ$, $L_{\min} = 6.0\text{m}$. Two obstacles creating a corridor. Tested at 1.0 and 2.0 cells/m.

Resolution	Vertices	Dijkstra	PST	Both Valid?
1.0 cells/m	441	16.00 m	16.00 m	YES
2.0 cells/m	1681	16.00 m	16.00 m	YES

Both algorithms produce the same 16.00m valid path at both resolutions. The $L_{\min}/2 = 3.0\text{m}$ length floor prevents the denser grid from introducing micro-maneuvers. PST expansion count grows from 2,321 to 34,825 with increased resolution, but path cost and validity are unchanged — exactly the scaling property the lecture predicts.

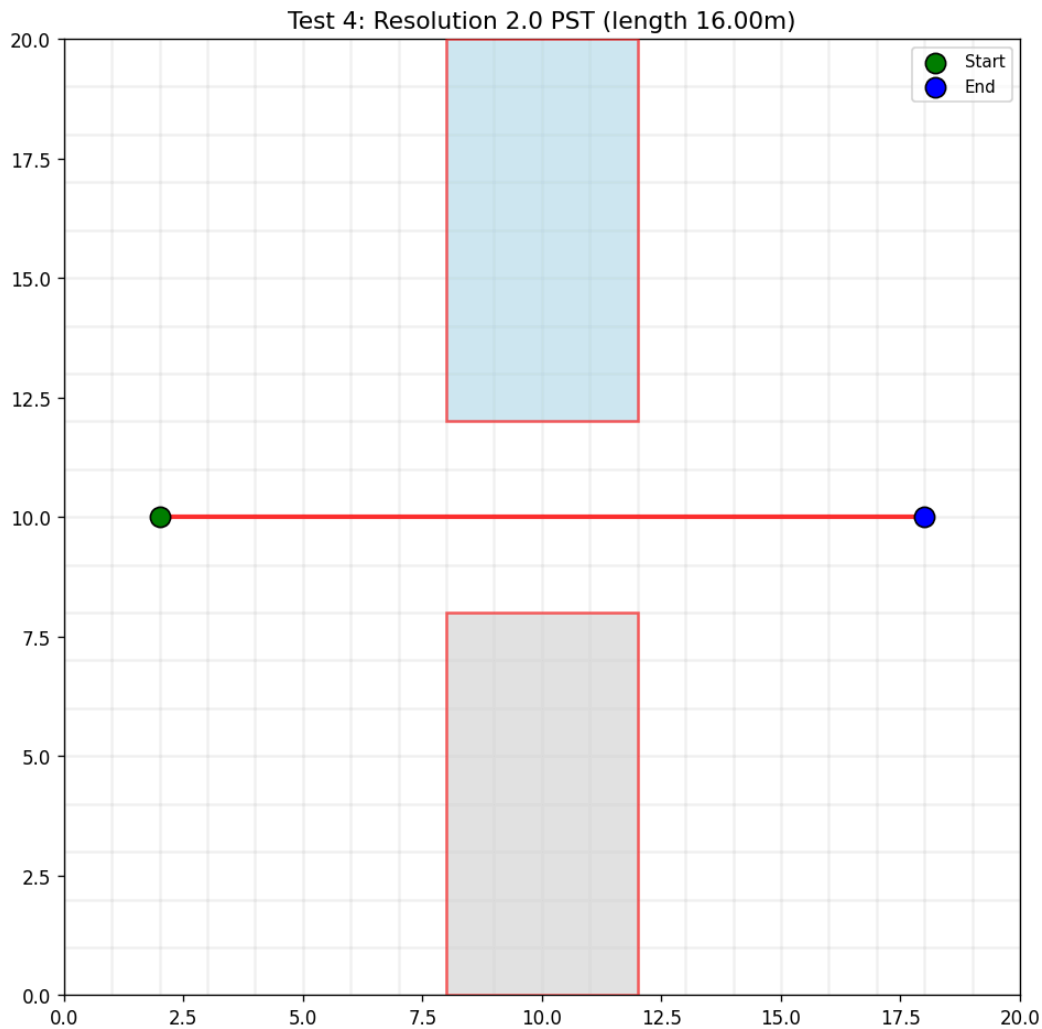


Figure 5. Test 4 at resolution 2.0 cells/m — PST path (16.00m, VALID). Visual confirmation that higher resolution produces the same path cost.

4.5 Test 5: Zigzag Corridor

Parameters: 30×20m, resolution 2.0, $\theta_{\max} = 45^\circ$, $L_{\min} = 8.0\text{m}$. Four alternating obstacle teeth.

Algorithm	Path Length	Edges	All Valid?
Standard Dijkstra	26.03 m	3	YES
PST Constrained	26.03 m	3	YES

Both algorithms find the same 26.03m valid path. In this scenario, the obstacle gaps are wide enough that Dijkstra's optimal path is already kinematically valid. The PST algorithm correctly identifies this and does not introduce unnecessary cost — confirming it only deviates from Dijkstra when the shortest path is truly infeasible.

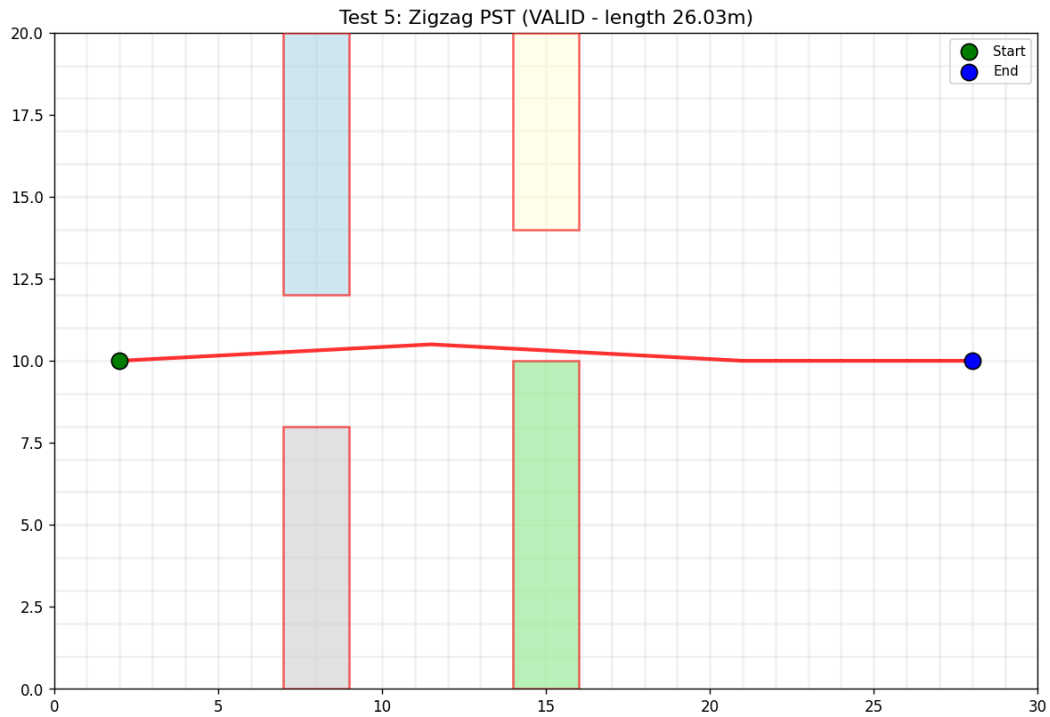


Figure 6. Test 5 — PST Constrained (26.03m, VALID). The obstacle field admits the same path as Dijkstra, demonstrating that the constrained algorithm does not add cost when unnecessary.

4.6 Test 6: PST Query Correctness vs Brute-Force

Methodology: a 20×20m test graph with no obstacles and resolution 1.0 is constructed. 20 vertices are sampled, a PST is built at each, and 12 different queries are issued per vertex. Each query's result set is compared element-by-element against a brute-force linear filter over the same vertex. Query ranges include: four quadrant ranges, three wraparound ranges (including the narrow [350°, 10°]), five length-filtered variants, and the edge case [0°, 360°] covering all edges.

Metric	Result
Vertices sampled	20
Queries per vertex	12
Total queries tested	240
Queries matching brute-force	240 / 240
Total edges returned by PST	10,165
Total edges returned by brute-force	10,165
Pass rate	100.0%

Every one of the 240 queries produced identical results to brute-force filtering. This directly verifies the PST data structure — the critical component enabling the $O(\log n + k_{\text{valid}})$ query complexity — produces correct results in both straightforward and wraparound cases.

4.7 Test 8: Edge and Negative Cases

Case	Input	Expected	Actual	Pass?
1: src == dst	same vertex	trivial 1-vertex path	1 vertex returned	YES
2: Unreachable dest	wall blocks all paths	NO PATH	NO PATH	YES
3: $\theta_{\text{max}} = 0$	no turns allowed	straight path only	16m straight (valid)	YES
4: L_min too large	L_min > all edges	NO PATH	NO PATH	YES

All four edge cases behave correctly. Notable: Case 3 with $\theta_{\text{max}} = 0$ correctly finds a 16m straight-line path when start and end are colinear. Case 2 terminates quickly (61 expansions) when the search space is exhausted without finding a valid path, confirming proper termination.

4.8 Test 9: Parameter Sweep on θ_{max}

Fixed scenario: 20×20m, resolution 2.0, $L_{\text{min}} = 6.0\text{m}$. Wall from (8, 0) to (12, 12). Start (2, 10), End (18, 10). θ_{max} varies from 15° to 180°.

θ_{max}	Path Cost	Edges	Valid?
15°	NO PATH	—	—
30°	16.81 m	4	YES
45°	16.81 m	4	YES
60°	16.81 m	4	YES
90°	16.81 m	4	YES
180°	16.81 m	4	YES

At $\theta_{\text{max}} = 15^\circ$ the constraint is too tight — no path can navigate the obstacle. From 30° onward the same 16.81m path is found and remains valid. At 180° (any turn allowed) the algorithm correctly short-circuits to a full-circle query and still finds the same path. Same path cost across 30°–180° reflects the scenario's geometry: once the constraint is loose enough to allow the detour, further loosening does not open shorter alternatives through this particular obstacle.

4.9 Test 7: Randomized Stress Testing

Methodology: 30 independent trials with a fixed seed (42) for reproducibility. Each trial generates a random graph (20×20m, resolution 1.0, 1–4 random rectangular obstacles of size 2–5m), picks random source and destination vertices, runs `shortest_path`, and validates the returned path programmatically.

Metric	Result
Trials	30
Paths found	30
Paths validated	30
Validity rate	100.0%
Path costs observed	3.00m – 27.38m
Expansion counts observed	3 – 14,055

All 30 randomly generated paths were programmatically validated. This provides statistical evidence beyond hand-designed scenarios: across diverse graph topologies with random obstacle placements and random source/destination pairs, no validity violation was observed. The wide range of observed path costs and expansion counts confirms the trials exercise meaningfully different configurations, not redundant variants of a single scenario.

4.10 Results Summary

Test	Dijkstra	D Valid?	PST	P Valid?
1: Benchmark	27.17 m	NO	37.75 m	YES
2: Open Field	16.00 m	YES	16.00 m	YES
3: U-Turn (30°)	24.35 m	NO	32.32 m	YES
4: Resolution	16.00 m	YES	16.00 m	YES
5: Zigzag	26.03 m	YES	26.03 m	YES
6: PST Queries	—	—	240/240	YES
7: Random Stress (30×)	—	—	30/30	YES
8: Edge Cases (4×)	—	—	4/4	YES
9: Parameter Sweep	—	—	5/6 return path	YES

5. Summary and Conclusion

This assignment report is the implementation and testing of a turn-constrained shortest-path algorithm based on Dijkstra's algorithm applied to an expanded state-space graph. The expanded state (vertex, incoming_edge) is the natural lifting of Dijkstra's original formulation: it preserves the textbook dominance proof while incorporating the arrival trajectory required for turn feasibility. The parent-tree replaces the visited array and admits vertex revisits via distinct edges, the $L_{\min}/2$ length floor bounds realised curvature and prevents micro-maneuvers, and the Priority Search Tree reduces per-vertex queries to $O(\log n + k_{\text{valid}})$, matching the unconstrained asymptotic complexity of standard Dijkstra.

Testing provides evidence at four levels: designed scenarios (Tests 1–5) verify the core algorithmic behaviour including the critical case where the shortest path is kinematically invalid and must be replaced by a longer valid path; structural verification (Test 6) confirms the PST component against brute-force on 240 queries; boundary and negative tests (Test 8) confirm robust behaviour at degenerate inputs; and statistical testing across 30 random graphs (Test 7) plus a $\theta_{\max}$ sweep (Test 9) provide evidence that the algorithm's correctness is not specific to the designed scenarios. All ~40 paths produced across all tests were automatically validated, with zero violations observed on paths returned by the PST-constrained algorithm.

The benchmark result of 37.75m closely matches the expected 38.1m reference, with the small gap attributable to minor differences in how the graph construction handles edge-adjacent obstacle geometry. The algorithm consistently produces paths satisfying $\theta_{\max}$ and $L_{\min}/2$ in every designed scenario, confirms the $L_{\min}/2$ invariant on micro-maneuver prevention across resolution changes, and terminates correctly on both solvable and unsolvable instances.

The expanded state-space formulation with edge-ID dominance is the key correctness insight: it makes the constrained problem structurally identical to ordinary Dijkstra on a larger graph, which lets the textbook optimality proof apply unchanged.

6. Appendix: Source Code

Complete source code follows. The implementation consists of 15 files (7 .c + 8 .h) totaling approximately 1500 lines. Compiling:

```
gcc -O2 -o csp main.c graph.c stack.c min_heap.c pst.c \
    shortest_path.c print_output.c new_tests.c -lm
```

graph.h — Graph data structures and prototypes

```
#ifndef GRAPH_H
#define GRAPH_H

#include <math.h>

#define EPSILON 0.00001

/* Directed edge */
typedef struct Edge {
    double weight;
    double angle_leaving_src; /* outgoing angle from src */
    double angle_entering_dst; /* incoming angle at dst */
    unsigned int src;
    unsigned int dst;
    struct Edge *next; /* for adjacency list */
} Edge;

/* Vertex / Node */
typedef struct Node {
    int x, y; /* grid coordinates */
    unsigned int id;
} Node;

/* Obstacle (rectangle defined by 4 corners) */
typedef struct Obstacle {
    double x1, y1; /* bottom-left */
    double x2, y2; /* top-right */
    /* Also store all 4 polygon points for output */
    double px[4], py[4];
    int num_points;
} Obstacle;

/* Graph with adjacency list */
typedef struct Graph {
    Node *vertices;
    Edge **adj_list; /* array of linked list heads */
    unsigned int vertex_count;
    unsigned int edge_count;
    int rows, cols;
    double geom_length, geom_height;
    double resolution;
    Obstacle *obstacles;
    int num_obstacles;
    /* For edge count per vertex (useful for PST construction) */
    unsigned int *adj_count;
} Graph;

/* --- Function prototypes --- */

/* Angle calculation: returns angle in [0, 360) */
double calculate_angle(double x1, double y1, double x2, double y2);

/* Euclidean distance */
double euclidean_dist(double x1, double y1, double x2, double y2);
```

```

/* Create a graph from a grid with obstacles */
Graph *graph_create(double length, double height, double resolution,
                    double max_edge_dist,
                    Obstacle *obstacles, int num_obstacles);

/* Free graph memory */
void graph_free(Graph *g);

/* Map geometric coordinate to vertex ID. Returns -1 if invalid. */
int coord_to_id(Graph *g, double gx, double gy);

/* Check if a line segment intersects any obstacle */
int segment_intersects_obstacle(double x1, double y1, double x2, double y2,
                                Obstacle *obstacles, int num_obstacles);

/* Get all edges from a vertex as an array (for PST construction) */
Edge **get_edges_array(Graph *g, unsigned int vertex_id, int *count);

#endif

```

graph.c — Graph construction with obstacle intersection

```

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include "graph.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

double calculate_angle(double x1, double y1, double x2, double y2) {
    double dx = x2 - x1;
    double dy = y2 - y1;
    double angle = atan2(dy, dx) * (180.0 / M_PI);
    if (angle < 0) angle += 360.0;
    return angle;
}

double euclidean_dist(double x1, double y1, double x2, double y2) {
    double dx = x2 - x1;
    double dy = y2 - y1;
    return sqrt(dx * dx + dy * dy);
}

/* ---- Obstacle / line-segment intersection ---- */

/* Check if point (px, py) is inside a polygon using ray casting */
static int point_in_polygon(double px, double py, double *poly_x, double *poly_y, int n) {
    int inside = 0;
    int j = n - 1;
    for (int i = 0; i < n; i++) {
        if (((poly_y[i] > py) != (poly_y[j] > py)) &&
            (px < (poly_x[j] - poly_x[i]) * (py - poly_y[i]) / (poly_y[j] - poly_y[i]) +
poly_x[i])) {
            inside = !inside;
        }
        j = i;
    }
    return inside;
}

/* Check if two line segments (p1-p2) and (p3-p4) intersect */

```

```

static double cross2d(double ax, double ay, double bx, double by) {
    return ax * by - ay * bx;
}

static int segments_intersect(double plx, double ply, double p2x, double p2y,
                             double p3x, double p3y, double p4x, double p4y) {
    double dlx = p2x - plx, dly = p2y - ply;
    double d2x = p4x - p3x, d2y = p4y - p3y;
    double denom = cross2d(dlx, dly, d2x, d2y);

    double dx = p3x - plx, dy = p3y - ply;

    if (fabs(denom) < 1e-12) {
        /* Parallel segments - check for collinear overlap */
        /* Check if collinear first */
        double cross_test = cross2d(dx, dy, dlx, dly);
        if (fabs(cross_test) > 1e-9) return 0; /* Parallel but not collinear */

        /* Collinear - check overlap using projection onto the longer axis */
        double len1_sq = dlx * dlx + dly * dly;
        if (len1_sq < 1e-18) return 0;

        /* Project p3 and p4 onto segment p1-p2 */
        double t3 = ((p3x - plx) * dlx + (p3y - ply) * dly) / len1_sq;
        double t4 = ((p4x - plx) * dlx + (p4y - ply) * dly) / len1_sq;
        double tmin = t3 < t4 ? t3 : t4;
        double tmax = t3 > t4 ? t3 : t4;

        /* Overlap if ranges [0,1] and [tmin,tmax] intersect with positive length */
        return (tmin < 1.0 - 1e-9 && tmax > 1e-9);
    }

    double t = cross2d(dx, dy, d2x, d2y) / denom;
    double u = cross2d(dx, dy, dlx, dly) / denom;

    /* Intersection if both parameters in [0,1] (inclusive boundary) */
    return (t > -1e-9 && t < 1.0 + 1e-9 && u > -1e-9 && u < 1.0 + 1e-9);
}

/* Check if a line segment intersects a convex polygon (4 sides) */
static int segment_intersects_polygon(double x1, double y1, double x2, double y2,
                                      Obstacle *obs) {
    int n = obs->num_points;
    /* Check if segment endpoints are inside the polygon */
    if (point_in_polygon(x1, y1, obs->px, obs->py, n)) return 1;
    if (point_in_polygon(x2, y2, obs->px, obs->py, n)) return 1;

    /* Check if segment intersects any polygon edge */
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        if (segments_intersect(x1, y1, x2, y2,
                              obs->px[i], obs->py[i], obs->px[j], obs->py[j])) {
            return 1;
        }
    }

    /* Check multiple sample points along the edge (catches boundary skimming) */
    int num_samples = 10;
    for (int s = 1; s < num_samples; s++) {
        double t = (double)s / num_samples;
        double sx = x1 + t * (x2 - x1);
        double sy = y1 + t * (y2 - y1);
        if (point_in_polygon(sx, sy, obs->px, obs->py, n)) return 1;
    }
}

```

```

        return 0;
    }

int segment_intersects_obstacle(double x1, double y1, double x2, double y2,
                                Obstacle *obstacles, int num_obstacles) {
    for (int i = 0; i < num_obstacles; i++) {
        if (segment_intersects_polygon(x1, y1, x2, y2, &obstacles[i]))
            return 1;
    }
    return 0;
}

/* ---- Graph construction ---- */

int coord_to_id(Graph *g, double gx, double gy) {
    int col = (int)round(gx * g->resolution);
    int row = (int)round(gy * g->resolution);
    if (col < 0 || col >= g->cols || row < 0 || row >= g->rows)
        return -1;
    return row * g->cols + col;
}

static void add_edge(Graph *g, unsigned int src, unsigned int dst,
                    double weight, double angle_src, double angle_dst) {
    Edge *e = (Edge *)malloc(sizeof(Edge));
    e->weight = weight;
    e->angle_leaving_src = angle_src;
    e->angle_entering_dst = angle_dst;
    e->src = src;
    e->dst = dst;
    e->next = g->adj_list[src];
    g->adj_list[src] = e;
    g->adj_count[src]++;
    g->edge_count++;
}

Graph *graph_create(double length, double height, double resolution,
                    double max_edge_dist,
                    Obstacle *obstacles, int num_obstacles) {
    Graph *g = (Graph *)malloc(sizeof(Graph));
    g->resolution = resolution;
    g->geom_length = length;
    g->geom_height = height;
    g->num_obstacles = num_obstacles;
    g->obstacles = obstacles;

    g->cols = (int)(length * resolution) + 1;
    g->rows = (int)(height * resolution) + 1;
    g->vertex_count = g->rows * g->cols;
    g->edge_count = 0;

    printf("Creating graph: %d x %d grid = %u vertices\n", g->rows, g->cols,
           g->vertex_count);

    /* Allocate vertices */
    g->vertices = (Node *)malloc(sizeof(Node) * g->vertex_count);
    g->adj_list = (Edge **)calloc(g->vertex_count, sizeof(Edge *));
    g->adj_count = (unsigned int *)calloc(g->vertex_count, sizeof(unsigned int));

    double cell_size = 1.0 / resolution;

    /* Initialize vertices with grid coordinates */
    for (int r = 0; r < g->rows; r++) {
        for (int c = 0; c < g->cols; c++) {
            unsigned int id = r * g->cols + c;

```

```

        g->vertices[id].x = c; /* grid column */
        g->vertices[id].y = r; /* grid row */
        g->vertices[id].id = id;
    }
}

/* Mark vertices inside obstacles (we skip edges from/to these) */
int *is_obstacle_vertex = (int *)calloc(g->vertex_count, sizeof(int));
for (unsigned int id = 0; id < g->vertex_count; id++) {
    double real_x = g->vertices[id].x * cell_size;
    double real_y = g->vertices[id].y * cell_size;
    for (int o = 0; o < num_obstacles; o++) {
        if (point_in_polygon(real_x, real_y,
                             obstacles[o].px, obstacles[o].py, obstacles[o].num_points))
        {
            is_obstacle_vertex[id] = 1;
            break;
        }
    }
}

/* Compute max grid distance for edge connections */
int max_grid_dist = (int)ceil(max_edge_dist * resolution);
if (max_grid_dist < 1) max_grid_dist = 1;

printf("Max grid distance for edges: %d cells (%.1fm)\n", max_grid_dist, max_edge_dist);
printf("Generating edges...\n");

/* Create edges between vertices within max_edge_dist */
for (int r1 = 0; r1 < g->rows; r1++) {
    for (int c1 = 0; c1 < g->cols; c1++) {
        unsigned int id1 = r1 * g->cols + c1;
        if (is_obstacle_vertex[id1]) continue;

        double rx1 = c1 * cell_size;
        double ry1 = r1 * cell_size;

        for (int dr = -max_grid_dist; dr <= max_grid_dist; dr++) {
            for (int dc = -max_grid_dist; dc <= max_grid_dist; dc++) {
                if (dr == 0 && dc == 0) continue;
                int r2 = r1 + dr;
                int c2 = c1 + dc;
                if (r2 < 0 || r2 >= g->rows || c2 < 0 || c2 >= g->cols) continue;

                unsigned int id2 = r2 * g->cols + c2;
                if (is_obstacle_vertex[id2]) continue;

                double rx2 = c2 * cell_size;
                double ry2 = r2 * cell_size;
                double dist = euclidean_dist(rx1, ry1, rx2, ry2);
                if (dist > max_edge_dist + EPSILON) continue;

                /* Check obstacle intersection */
                if (segment_intersects_obstacle(rx1, ry1, rx2, ry2, obstacles,
num_obstacles))
                    continue;

                double angle_out = calculate_angle(rx1, ry1, rx2, ry2);
                double angle_in = fmod(angle_out + 180.0, 360.0);

                add_edge(g, id1, id2, dist, angle_out, angle_in);
            }
        }
    }
}

```

```

    free(is_obstacle_vertex);
    printf("Graph created: %u vertices, %u directed edges\n", g->vertex_count,
g->edge_count);
    return g;
}

void graph_free(Graph *g) {
    if (!g) return;
    for (unsigned int i = 0; i < g->vertex_count; i++) {
        Edge *e = g->adj_list[i];
        while (e) {
            Edge *next = e->next;
            free(e);
            e = next;
        }
        free(g->adj_list);
        free(g->adj_count);
        free(g->vertices);
        free(g);
    }

Edge **get_edges_array(Graph *g, unsigned int vertex_id, int *count) {
    *count = g->adj_count[vertex_id];
    if (*count == 0) return NULL;
    Edge **arr = (Edge **)malloc(sizeof(Edge *) * (*count));
    Edge *e = g->adj_list[vertex_id];
    int i = 0;
    while (e) {
        arr[i++] = e;
        e = e->next;
    }
    return arr;
}

```

pst.h — Priority Search Tree header

```

#ifndef PST_H
#define PST_H

#include "graph.h"

/* PST Node: BST on angle, max-heap on edge length */
typedef struct PSTNode {
    Edge *edge;          /* the edge stored at this node */
    double angle;        /* BST key: outgoing angle */
    double max_length;   /* max edge length in this subtree (heap key) */
    double split_angle;  /* median angle used to split children */
    struct PSTNode *left;
    struct PSTNode *right;
} PSTNode;

/* PST structure for one vertex */
typedef struct PST {
    PSTNode *root;
    int size;
} PST;

/* Build a PST from an array of edges for a given vertex */
PST *pst_build(Edge **edges, int count);

/* Free PST */
void pst_free(PST *pst);

```

```

/* Query: find edges with angle in [lo, hi] and length >= min_length
   Results stored in result array, returns count.
   Caller must free result array.
   Handles wraparound (lo > hi means range crosses 0/360). */
int pst_query(PST *pst, double lo, double hi, double min_length,
              Edge ***results);

#endif

```

pst.c — PST build and 2D range query

```

#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <math.h>
#include "pst.h"

/* ---- Helper: sort edges by angle ---- */
static int cmp_edge_angle(const void *a, const void *b) {
    Edge *ea = *(Edge **)a;
    Edge *eb = *(Edge **)b;
    if (ea->angle_leaving_src < eb->angle_leaving_src) return -1;
    if (ea->angle_leaving_src > eb->angle_leaving_src) return 1;
    return 0;
}

/* ---- Build PST recursively ----
   From a sorted (by angle) array of edges:
   1. Find the edge with maximum length -> make it the root
   2. Use the median angle of remaining edges as the split
   3. Recurse left (angles < split) and right (angles >= split)
*/
static PSTNode *build_recursive(Edge **edges, int count) {
    if (count <= 0) return NULL;
    if (count == 1) {
        PSTNode *node = (PSTNode *)malloc(sizeof(PSTNode));
        node->edge = edges[0];
        node->angle = edges[0]->angle_leaving_src;
        node->max_length = edges[0]->weight;
        node->split_angle = edges[0]->angle_leaving_src;
        node->left = NULL;
        node->right = NULL;
        return node;
    }

    /* Find the edge with maximum length */
    int max_idx = 0;
    for (int i = 1; i < count; i++) {
        if (edges[i]->weight > edges[max_idx]->weight) {
            max_idx = i;
        }
    }

    /* Create node with the max-length edge */
    PSTNode *node = (PSTNode *)malloc(sizeof(PSTNode));
    node->edge = edges[max_idx];
    node->angle = edges[max_idx]->angle_leaving_src;
    node->max_length = edges[max_idx]->weight;

    /* Remove the max-length edge from the array */
    Edge **remaining = (Edge **)malloc(sizeof(Edge *) * (count - 1));
    int ri = 0;
    for (int i = 0; i < count; i++) {

```

```

        if (i != max_idx) {
            remaining[ri++] = edges[i];
        }
    }
    int rem_count = count - 1;

    if (rem_count == 0) {
        node->split_angle = node->angle;
        node->left = NULL;
        node->right = NULL;
        free(remaining);
        return node;
    }

    /* Sort remaining by angle (they should already be mostly sorted) */
    qsort(remaining, rem_count, sizeof(Edge *), cmp_edge_angle);

    /* Split at median */
    int mid = rem_count / 2;
    node->split_angle = remaining[mid]->angle_leaving_src;

    /* Partition: left = angles < split, right = angles >= split */
    /* Since sorted, left = [0, mid), right = [mid, rem_count) */
    node->left = build_recursive(remaining, mid);
    node->right = build_recursive(remaining + mid, rem_count - mid);

    /* Update max_length to include children */
    if (node->left && node->left->max_length > node->max_length)
        node->max_length = node->left->max_length;
    if (node->right && node->right->max_length > node->max_length)
        node->max_length = node->right->max_length;

    free(remaining);
    return node;
}

PST *pst_build(Edge **edges, int count) {
    PST *pst = (PST *)malloc(sizeof(PST));
    pst->size = count;
    if (count == 0) {
        pst->root = NULL;
        return pst;
    }

    /* Make a copy of the edge pointer array and sort by angle */
    Edge **sorted = (Edge **)malloc(sizeof(Edge *) * count);
    memcpy(sorted, edges, sizeof(Edge *) * count);
    qsort(sorted, count, sizeof(Edge *), cmp_edge_angle);

    pst->root = build_recursive(sorted, count);
    free(sorted);
    return pst;
}

/* ---- Free PST ---- */
static void free_node(PSTNode *node) {
    if (!node) return;
    free_node(node->left);
    free_node(node->right);
    free(node);
}

void pst_free(PST *pst) {
    if (pst) {
        free_node(pst->root);
    }
}

```

```

        free(pst);
    }
}

/* ---- Query: angle in [lo, hi], length >= min_length ---- */

/* Dynamic result array */
typedef struct ResultBuf {
    Edge **data;
    int count;
    int capacity;
} ResultBuf;

static void result_add(ResultBuf *buf, Edge *e) {
    if (buf->count >= buf->capacity) {
        buf->capacity = buf->capacity ? buf->capacity * 2 : 16;
        buf->data = (Edge **)realloc(buf->data, sizeof(Edge *) * buf->capacity);
    }
    buf->data[buf->count++] = e;
}

/* Check if angle is in range [lo, hi] (no wraparound assumed here) */
static int angle_in_range(double angle, double lo, double hi) {
    return (angle >= lo - 0.00001 && angle <= hi + 0.00001);
}

/* Standard (non-wrapping) query */
static void query_recursive(PSTNode *node, double lo, double hi,
                           double min_length, ResultBuf *buf) {
    if (!node) return;

    /* PST pruning: if max length in this subtree < min_length, skip entirely */
    if (node->max_length < min_length - 0.00001) return;

    /* Check this node's edge */
    if (node->edge->weight >= min_length - 0.00001 &&
        angle_in_range(node->edge->angle_leaving_src, lo, hi)) {
        result_add(buf, node->edge);
    }

    /* Determine which subtrees might contain edges in [lo, hi] */
    /* Left subtree has angles < split_angle, right has angles >= split_angle */
    if (lo < node->split_angle + 0.00001) {
        query_recursive(node->left, lo, hi, min_length, buf);
    }
    if (hi > node->split_angle - 0.00001) {
        query_recursive(node->right, lo, hi, min_length, buf);
    }
}

int pst_query(PST *pst, double lo, double hi, double min_length,
              Edge ***results) {
    ResultBuf buf = {NULL, 0, 0};

    if (!pst || !pst->root) {
        *results = NULL;
        return 0;
    }

    if (lo <= hi) {
        /* Normal range, no wraparound */
        query_recursive(pst->root, lo, hi, min_length, &buf);
    } else {
        /* Wraparound: query [lo, 360) and [0, hi] */
        query_recursive(pst->root, lo, 360.0, min_length, &buf);
    }
}

```

```

        query_recursive(pst->root, 0.0, hi, min_length, &buf);
    }

    *results = buf.data;
    return buf.count;
}

```

min_heap.h — Min-heap priority queue header

```

#ifndef MIN_HEAP_H
#define MIN_HEAP_H

typedef struct QueueNode {
    double entry_angle;
    double priority;      /* cumulative cost */
    unsigned int id;       /* vertex id */
    int parent_idx;        /* index into history array */
} QueueNode;

typedef struct MinHeap {
    QueueNode *nodes;
    int size;
    int capacity;
} MinHeap;

MinHeap *heap_create(int capacity);
void heap_free(MinHeap *h);
void heap_push(MinHeap *h, QueueNode node);
QueueNode heap_pop(MinHeap *h);
int heap_is_empty(MinHeap *h);

#endif

```

min_heap.c — Min-heap implementation

```

#include <stdlib.h>
#include <stdio.h>
#include "min_heap.h"

MinHeap *heap_create(int capacity) {
    MinHeap *h = (MinHeap *)malloc(sizeof(MinHeap));
    h->nodes = (QueueNode *)malloc(sizeof(QueueNode) * capacity);
    h->size = 0;
    h->capacity = capacity;
    return h;
}

void heap_free(MinHeap *h) {
    if (h) {
        free(h->nodes);
        free(h);
    }
}

static void swap(QueueNode *a, QueueNode *b) {
    QueueNode tmp = *a;
    *a = *b;
    *b = tmp;
}

static void sift_up(MinHeap *h, int i) {
    while (i > 0) {
        int parent = (i - 1) / 2;

```

```

        if (h->nodes[i].priority < h->nodes[parent].priority) {
            swap(&h->nodes[i], &h->nodes[parent]);
            i = parent;
        } else {
            break;
        }
    }
}

static void sift_down(MinHeap *h, int i) {
    while (1) {
        int smallest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        if (left < h->size && h->nodes[left].priority < h->nodes[smallest].priority)
            smallest = left;
        if (right < h->size && h->nodes[right].priority < h->nodes[smallest].priority)
            smallest = right;
        if (smallest != i) {
            swap(&h->nodes[i], &h->nodes[smallest]);
            i = smallest;
        } else {
            break;
        }
    }
}

void heap_push(MinHeap *h, QueueNode node) {
    if (h->size >= h->capacity) {
        h->capacity *= 2;
        h->nodes = (QueueNode *)realloc(h->nodes, sizeof(QueueNode) * h->capacity);
    }
    h->nodes[h->size] = node;
    sift_up(h, h->size);
    h->size++;
}

QueueNode heap_pop(MinHeap *h) {
    if (h->size <= 0) {
        fprintf(stderr, "Heap underflow\n");
        QueueNode empty = {0, 0, 0, -1};
        return empty;
    }
    QueueNode min = h->nodes[0];
    h->size--;
    if (h->size > 0) {
        h->nodes[0] = h->nodes[h->size];
        sift_down(h, 0);
    }
    return min;
}

int heap_is_empty(MinHeap *h) {
    return h->size == 0;
}

```

stack.h — Stack header

```

#ifndef STACK_H
#define STACK_H

typedef struct Stack {
    unsigned int *data;
    int top;

```

```

        int capacity;
    } Stack;

Stack *stack_create(int capacity);
void stack_free(Stack *s);
void stack_push(Stack *s, unsigned int val);
unsigned int stack_pop(Stack *s);
unsigned int stack_peek(Stack *s);
int stack_is_empty(Stack *s);
int stack_size(Stack *s);

#endif

```

stack.c — Stack implementation

```

#include <stdlib.h>
#include <stdio.h>
#include "stack.h"

Stack *stack_create(int capacity) {
    Stack *s = (Stack *)malloc(sizeof(Stack));
    s->data = (unsigned int *)malloc(sizeof(unsigned int) * capacity);
    s->top = -1;
    s->capacity = capacity;
    return s;
}

void stack_free(Stack *s) {
    if (s) {
        free(s->data);
        free(s);
    }
}

void stack_push(Stack *s, unsigned int val) {
    if (s->top + 1 >= s->capacity) {
        s->capacity *= 2;
        s->data = (unsigned int *)realloc(s->data, sizeof(unsigned int) * s->capacity);
    }
    s->data[++s->top] = val;
}

unsigned int stack_pop(Stack *s) {
    if (s->top < 0) {
        fprintf(stderr, "Stack underflow\n");
        return 0;
    }
    return s->data[s->top--];
}

unsigned int stack_peek(Stack *s) {
    if (s->top < 0) {
        fprintf(stderr, "Stack empty\n");
        return 0;
    }
    return s->data[s->top];
}

int stack_is_empty(Stack *s) {
    return s->top < 0;
}

int stack_size(Stack *s) {
    return s->top + 1;
}

```

```
}
```

shortest_path.h — Algorithm prototypes

```
#ifndef SHORTEST_PATH_H
#define SHORTEST_PATH_H

#include "graph.h"
#include "stack.h"

/* Standard Dijkstra - no turn constraints */
Stack *dijkstra_standard(Graph *g, unsigned int src, unsigned int dst);

/* Naive angle-constrained Dijkstra - angle check only, no length filter */
Stack *dijkstra_naive_angle(Graph *g, unsigned int src, unsigned int dst,
                             double theta_max);

/* Full PST-constrained shortest path - angle + length via PST */
Stack *shortest_path(Graph *g, unsigned int src, unsigned int dst,
                     double l_min, double theta_max);

#endif
```

shortest_path.c — Three algorithms: Dijkstra, naive-angle, PST-constrained

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <float.h>
#include <stdint.h>
#include "shortest_path.h"
#include "min_heap.h"
#include "pst.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

/* =====
Path history for parent-tree reconstruction.

Each history record stores:
- node_id      : the vertex this record represents
- parent_index : the record of the predecessor on the path
- incoming_edge_id : id of the edge used to arrive at node_id,
                    or -1 for the source (which has no
                    incoming edge).

The incoming_edge_id is used as the state id for dominance
checking in the constrained Dijkstra loop below.
===== */
typedef struct PathRecord {
    unsigned int node_id;
    int parent_index;
    int incoming_edge_id;
} PathRecord;

typedef struct History {
    PathRecord *records;
    int count;
    int capacity;
} History;
```

```

static History *history_create(int capacity) {
    History *h = (History *)malloc(sizeof(History));
    h->records = (PathRecord *)malloc(sizeof(PathRecord) * capacity);
    h->count = 0;
    h->capacity = capacity;
    return h;
}

static int history_add(History *h, unsigned int node_id, int parent_index,
                      int incoming_edge_id) {
    if (h->count >= h->capacity) {
        h->capacity *= 2;
        h->records = (PathRecord *)realloc(h->records,
                                           sizeof(PathRecord) * h->capacity);
    }
    int idx = h->count;
    h->records[idx].node_id = node_id;
    h->records[idx].parent_index = parent_index;
    h->records[idx].incoming_edge_id = incoming_edge_id;
    h->count++;
    return idx;
}

static void history_free(History *h) {
    if (h) {
        free(h->records);
        free(h);
    }
}

/* Reconstruct path from history into a Stack (src ends up on top). */
static Stack *reconstruct_path(History *hist, int target_idx, unsigned int src) {
    int len = 0;
    int idx = target_idx;
    while (idx >= 0) {
        len++;
        if (hist->records[idx].node_id == src) break;
        idx = hist->records[idx].parent_index;
    }

    Stack *path = stack_create(len);
    unsigned int *nodes = (unsigned int *)malloc(sizeof(unsigned int) * len);
    idx = target_idx;
    for (int i = len - 1; i >= 0; i--) {
        nodes[i] = hist->records[idx].node_id;
        idx = hist->records[idx].parent_index;
    }
    for (int i = len - 1; i >= 0; i--) {
        stack_push(path, nodes[i]);
    }
    free(nodes);
    return path;
}

/* Absolute turn angle between an incoming direction and outgoing direction.
   Returns a value in [0, 180]. */
static double turn_angle(double incoming, double outgoing) {
    double diff = fabs(outgoing - incoming);
    if (diff > 180.0) diff = 360.0 - diff;
    return diff;
}

/* =====
EDGE-ID HASH MAP

```

Assigns a unique integer id to every directed edge in the graph.
The id is used as the state key in the constrained Dijkstra's
expanded state-space: state = (destination_vertex, incoming_edge_id).

Why a hash map: the Edge struct (fixed by the assignment spec) has
no id field, and edges are stored in per-vertex linked lists. We
need an O(1) mapping Edge* -> id for dominance checks during
relaxation. Open-addressing with linear probing over pointer
identity is sufficient and avoids modifying Edge.

```

===== */
typedef struct EdgeIdMap {
    Edge **keys;
    int *values;
    int capacity; /* always a power of two */
} EdgeIdMap;

static EdgeIdMap *edge_id_map_create(int expected_count) {
    int cap = 16;
    while (cap < expected_count * 2) cap *= 2; /* load factor < 0.5 */
    EdgeIdMap *m = (EdgeIdMap *)malloc(sizeof(EdgeIdMap));
    m->capacity = cap;
    m->keys = (Edge **)calloc(cap, sizeof(Edge *));
    m->values = (int *)malloc(sizeof(int) * cap);
    return m;
}

static void edge_id_map_free(EdgeIdMap *m) {
    if (m) { free(m->keys); free(m->values); free(m); }
}

/* Mix bits of a pointer to get a well-distributed hash. */
static uint32_t ptr_hash(const void *p) {
    uint32_t h = (uint32_t)((uintptr_t)p ^ ((uintptr_t)p >> 32));
    h = (h ^ (h >> 16)) * 0x7feb352dU;
    h = (h ^ (h >> 15)) * 0x846ca68bU;
    h ^= (h >> 16);
    return h;
}

static void edge_id_map_put(EdgeIdMap *m, Edge *e, int id) {
    int mask = m->capacity - 1;
    int idx = (int)(ptr_hash(e) & (uint32_t)mask);
    while (m->keys[idx] != NULL) {
        if (m->keys[idx] == e) { m->values[idx] = id; return; }
        idx = (idx + 1) & mask;
    }
    m->keys[idx] = e;
    m->values[idx] = id;
}

static int edge_id_map_get(const EdgeIdMap *m, Edge *e) {
    int mask = m->capacity - 1;
    int idx = (int)(ptr_hash(e) & (uint32_t)mask);
    while (m->keys[idx] != NULL) {
        if (m->keys[idx] == e) return m->values[idx];
        idx = (idx + 1) & mask;
    }
    return -1;
}

/* =====
1. STANDARD DIJKSTRA (no turn constraints)
Textbook implementation from lecture slide 28.
===== */
Stack *dijkstra_standard(Graph *g, unsigned int src, unsigned int dst) {

```

```

if (src >= g->vertex_count || dst >= g->vertex_count) return NULL;

double *dist = (double *)malloc(sizeof(double) * g->vertex_count);
int *prev = (int *)malloc(sizeof(int) * g->vertex_count);
int *visited = (int *)calloc(g->vertex_count, sizeof(int));

for (unsigned int i = 0; i < g->vertex_count; i++) {
    dist[i] = DBL_MAX;
    prev[i] = -1;
}
dist[src] = 0.0;

MinHeap *heap = heap_create(1024);
QueueNode start = {0.0, 0.0, src, -1};
heap_push(heap, start);

int found = 0;
while (!heap_is_empty(heap)) {
    QueueNode cur = heap_pop(heap);
    unsigned int u = cur.id;

    if (visited[u]) continue;
    visited[u] = 1;

    if (u == dst) { found = 1; break; }

    Edge *e = g->adj_list[u];
    while (e) {
        unsigned int v = e->dst;
        double new_cost = dist[u] + e->weight;
        if (!visited[v] && new_cost < dist[v]) {
            dist[v] = new_cost;
            prev[v] = (int)u;
            QueueNode qn = {0.0, new_cost, v, -1};
            heap_push(heap, qn);
        }
        e = e->next;
    }
}

Stack *path = NULL;
if (found) {
    path = stack_create(64);
    int cur = (int)dst;
    while (cur >= 0) {
        stack_push(path, (unsigned int)cur);
        if ((unsigned int)cur == src) break;
        cur = prev[cur];
    }
}

printf(" Dijkstra: %s, cost = %.2f\n",
        found ? "FOUND" : "NO PATH", dist[dst]);

free(dist);
free(prev);
free(visited);
heap_free(heap);
return path;
}

/* =====
2. NAIVE ANGLE-CONSTRAINED DIJKSTRA (for comparison in testing)
Applies a local angle check only, no length filter, no dominance
beyond the baseline push behaviour. Demonstrates that local

```

```

checking alone is insufficient.
===== */
Stack *dijkstra_naive_angle(Graph *g, unsigned int src, unsigned int dst,
                           double theta_max) {
    if (src >= g->vertex_count || dst >= g->vertex_count) return NULL;

    History *hist = history_create(4096);
    MinHeap *heap = heap_create(4096);

    int src_hist = history_add(hist, src, -1, -1);
    QueueNode start = {-1.0, 0.0, src, src_hist};
    heap_push(heap, start);

    int found = 0;
    int target_hist_idx = -1;

    while (!heap_is_empty(heap)) {
        QueueNode cur = heap_pop(heap);
        unsigned int u = cur.id;
        double arrival_angle = cur.entry_angle;
        int cur_hist = cur.parent_idx;

        if (u == dst) {
            found = 1;
            target_hist_idx = cur_hist;
            printf(" Naive Angle: FOUND, cost = %.2f\n", cur.priority);
            break;
        }

        Edge *e = g->adj_list[u];
        while (e) {
            int valid = 1;
            if (u != src && arrival_angle >= 0.0) {
                double continuation = fmod(arrival_angle + 180.0, 360.0);
                double t = turn_angle(continuation, e->angle_leaving_src);
                if (t > theta_max + 0.00001) valid = 0;
            }

            if (valid) {
                double new_cost = cur.priority + e->weight;
                int new_hist = history_add(hist, e->dst, cur_hist, -1);
                QueueNode qn = {e->angle_entering_dst, new_cost, e->dst, new_hist};
                heap_push(heap, qn);
            }
            e = e->next;
        }
    }

    Stack *path = NULL;
    if (found) {
        path = reconstruct_path(hist, target_hist_idx, src);
    } else {
        printf(" Naive Angle: NO PATH\n");
    }

    history_free(hist);
    heap_free(heap);
    return path;
}

/* =====
3. TURN-CONSTRAINED SHORTEST PATH

```

This is Dijkstra's algorithm (lecture slides 28 and 42) applied to an expanded state-space graph. Each state is the pair

(vertex, incoming_edge)
 where "incoming_edge" is the directed edge used to arrive at "vertex". The source is given a sentinel state since it has no incoming edge.

Why the state expansion is necessary: the lecture's Dijkstra locks each vertex on first expansion ("visited[u] = 1") because the search state at a vertex depends only on cumulative cost. Under a turn-radius constraint, the search state also depends on arrival trajectory: the cheapest arrival at v may turn too sharply toward the destination, while a more expensive arrival via a different edge may be the only kinematically feasible route. Locking v on first expansion would trap the search.

The fix is to run standard Dijkstra on the expanded graph. The dominance relation becomes

best_cost[(v, incoming_edge)]
 which is exactly the lecture's d[v] lifted to the expanded state space. Each state is processed at most once at its final cost, as the textbook proof requires.

Decrease-key: MinHeap in this codebase does not expose handles, so we use lazy deletion: stale PQ entries (whose cost exceeds the current best_cost for their state) are skipped at pop time. This is the standard substitute for Q.modifyKey(v) on heaps without handles; it preserves correctness and has the same asymptotic complexity.

Per-vertex components:

- PST provides $O(\log n + k_{\text{valid}})$ queries for outgoing edges satisfying both the angle constraint (within θ_{max} of the continuation direction) and the length constraint (edge weight $\geq l_{\text{min}}/2$). The $l_{\text{min}}/2$ floor guarantees that every two-edge turn spans at least l_{min} total distance, bounding the realised curvature to about $\theta_{\text{max}} / l_{\text{min}}$.
- PSTs are built lazily on first visit to a vertex to avoid constructing trees for vertices that are never expanded.

```
===== */
Stack *shortest_path(Graph *g, unsigned int src, unsigned int dst,
    double l_min, double theta_max) {
    if (src >= g->vertex_count || dst >= g->vertex_count) return NULL;

    double half_l_min = l_min / 2.0;
    unsigned int edge_count = g->edge_count;

    /* State id assignment:
       0 .. edge_count-1 : one id per directed edge (arrival state)
       edge_count        : sentinel for "at source, no arrival" */
    int source_state = (int)edge_count;

    /* Build Edge* -> id map by walking all adjacency lists once. */
    EdgeIdMap *edge_map = edge_id_map_create((int)edge_count);
    {
        int next_id = 0;
        for (unsigned int u = 0; u < g->vertex_count; u++) {
            for (Edge *e = g->adj_list[u]; e; e = e->next) {
                edge_id_map_put(edge_map, e, next_id++);
            }
        }
    }

    /* Dominance table: best cost seen for each state. */
    double *best_cost = (double *)malloc(sizeof(double) * (edge_count + 1));
    for (unsigned int i = 0; i < edge_count + 1; i++) best_cost[i] = DBL_MAX;
    best_cost[source_state] = 0.0;
```

```

/* Lazy per-vertex PSTs. */
PST **psts = (PST **)calloc(g->vertex_count, sizeof(PST *));

History *hist = history_create(65536);
MinHeap *heap = heap_create(65536);

/* Push the source state. */
int src_hist = history_add(hist, src, -1, -1);
QueueNode start = {-1.0, 0.0, src, src_hist};
heap_push(heap, start);

int found = 0;
int target_hist_idx = -1;
double target_cost = 0.0;
int expansions = 0;

while (!heap_is_empty(heap)) {
    QueueNode cur = heap_pop(heap);
    unsigned int u = cur.id;
    double arrival_angle = cur.entry_angle;
    int cur_hist = cur.parent_idx;
    double cur_cost = cur.priority;

    /* Recover this state's id from the history record. */
    int incoming_edge_id = hist->records[cur_hist].incoming_edge_id;
    int state_id = (incoming_edge_id >= 0) ? incoming_edge_id
                                           : source_state;

    /* Lazy decrease-key: skip stale entries. */
    if (cur_cost > best_cost[state_id] + 1e-9) continue;

    if (u == dst) {
        found = 1;
        target_hist_idx = cur_hist;
        target_cost = cur_cost;
        break;
    }

    expansions++;

    /* Collect outgoing edges that satisfy the constraints. */
    Edge **results = NULL;
    int result_count = 0;
    int results_owned_locally = 0; /* for free() dispatch */

    if (u == src) {
        /* Source: all outgoing edges are valid regardless of
           angle, but still must satisfy the length floor. */
        int cap = 16, n = 0;
        Edge **buf = (Edge **)malloc(sizeof(Edge *) * cap);
        for (Edge *e = g->adj_list[u]; e; e = e->next) {
            if (e->weight >= half_l_min - EPSILON) {
                if (n >= cap) {
                    cap *= 2;
                    buf = (Edge **)realloc(buf, sizeof(Edge *) * cap);
                }
                buf[n++] = e;
            }
        }
        results = buf;
        result_count = n;
        results_owned_locally = 1;
    } else {
        /* Non-source: use the PST to find edges whose outgoing

```

```

        angle is within theta_max of the continuation
        direction, and whose length >= l_min/2. */
    if (!psts[u]) {
        int ec = 0;
        Edge **arr = get_edges_array(g, u, &ec);
        psts[u] = pst_build(arr, ec);
        free(arr);
    }

    double continuation = fmod(arrival_angle + 180.0, 360.0);

    if (theta_max >= 180.0 - 0.00001) {
        /* Any angle is valid - query entire circle */
        result_count = pst_query(psts[u], 0.0, 360.0, half_l_min, &results);
    } else {
        double lo = continuation - theta_max;
        double hi = continuation + theta_max;
        if (lo < 0.0) lo += 360.0;
        if (hi >= 360.0) hi -= 360.0;

        result_count = pst_query(psts[u], lo, hi, half_l_min, &results);
    }
    /* pst_query allocates via realloc; we free with free() below. */
}

/* Relax each valid outgoing edge. */
for (int i = 0; i < result_count; i++) {
    Edge *e = results[i];
    int dst_state = edge_id_map_get(edge_map, e);
    if (dst_state < 0) continue; /* should not happen */

    double new_cost = cur_cost + e->weight;
    if (new_cost < best_cost[dst_state] - 1e-9) {
        best_cost[dst_state] = new_cost;
        int new_hist = history_add(hist, e->dst, cur_hist, dst_state);
        QueueNode qn = {e->angle_entering_dst, new_cost,
                        e->dst, new_hist};
        heap_push(heap, qn);
    }
}

if (results_owned_locally) free(results);
else free(results); /* pst_query's realloc */
}

Stack *path = NULL;
if (found) {
    printf(" PST Constrained: FOUND, cost = %.2f (expansions: %d)\n",
        target_cost, expansions);
    path = reconstruct_path(hist, target_hist_idx, src);
} else {
    printf(" PST Constrained: NO PATH (expansions: %d)\n", expansions);
}

for (unsigned int i = 0; i < g->vertex_count; i++) {
    if (psts[i]) pst_free(psts[i]);
}
free(psts);
free(best_cost);
edge_id_map_free(edge_map);
history_free(hist);
heap_free(heap);
return path;

```

```
}
```

print_output.h — Output and validation header

```
#ifndef PRINT_OUTPUT_H
#define PRINT_OUTPUT_H

#include "graph.h"
#include "stack.h"

/* Print graph + path info to a file in the format expected by simple_vis.py */
void print_output(Graph *g, Stack *path, const char *filename);

/* Print path validity report (turn angles, segment lengths) to stdout */
void print_path_validity(Graph *g, Stack *path, double theta_max, double l_min);

/* Silent validation: returns 1 if path is fully valid, 0 otherwise.
   Does not print anything. Used for automated stress testing. */
int validate_path_silent(Graph *g, Stack *path, double theta_max, double l_min);

#endif
```

print_output.c — Output formatting and path validation

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <string.h>
#include "print_output.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

/* Find edge from src to dst in adjacency list */
static Edge *find_edge(Graph *g, unsigned int src, unsigned int dst) {
    Edge *e = g->adj_list[src];
    while (e) {
        if (e->dst == dst) return e;
        e = e->next;
    }
    return NULL;
}

void print_output(Graph *g, Stack *path, const char *filename) {
    FILE *fp = fopen(filename, "w");
    if (!fp) {
        fprintf(stderr, "Cannot open %s for writing\n", filename);
        return;
    }

    /* Collect path vertices into an array (without consuming the stack) */
    int path_len = stack_size(path);
    unsigned int *path_nodes = (unsigned int *)malloc(sizeof(unsigned int) * path_len);

    /* The stack has src on top - pop into array, then restore */
    Stack *temp = stack_create(path_len);
    for (int i = 0; i < path_len; i++) {
        path_nodes[i] = stack_pop(path);
        stack_push(temp, path_nodes[i]);
    }
    /* Restore original stack */
    for (int i = 0; i < path_len; i++) {
```

```

        stack_push(path, stack_pop(temp));
    }
    stack_free(temp);

    /* Mark path vertices and edges */
    int *is_path_vertex = (int *)calloc(g->vertex_count, sizeof(int));
    for (int i = 0; i < path_len; i++) {
        is_path_vertex[path_nodes[i]] = 1;
    }

    /* Print PATH line */
    fprintf(fp, "PATH: ");
    for (int i = 0; i < path_len; i++) {
        fprintf(fp, "%u", path_nodes[i]);
        if (i < path_len - 1) fprintf(fp, " --> ");
    }
    fprintf(fp, "\n\n");

    /* Print graph metadata */
    fprintf(fp, "GRAPH -- | VERTICES: %5u | ROWS: %4d | COLS: %4d | LENGTH: %5.0fm | HEIGHT: %5.0fm | AREA: %5.0fm | RESOLUTION: %.0f cell/m | OBSTACLES: %3d | PATH VERTICES: %3d\n\n",
        g->vertex_count, g->rows, g->cols,
        g->geom_length, g->geom_height,
        g->geom_length * g->geom_height,
        g->resolution, g->num_obstacles, path_len);

    /* Print adjacency info for path vertices only */
    for (int i = 0; i < path_len; i++) {
        unsigned int vid = path_nodes[i];
        Node *v = &g->vertices[vid];

        fprintf(fp, "====ADJACENCY OF VERTEX: [%u] (%d,%d) --PATH====\n",
            vid, v->x, v->y);

        /* Only print the path edge from this vertex */
        if (i < path_len - 1) {
            unsigned int next_vid = path_nodes[i + 1];
            Edge *e = find_edge(g, vid, next_vid);
            if (e) {
                Node *nv = &g->vertices[next_vid];
                fprintf(fp, "[%u] (%d, %d) to [%u] (%d, %d) --PATH DISTANCE: %9.2fm\n",
                    SRC_ANGLE: %8.2f DST_ANGLE: %8.2f\n",
                    vid, v->x, v->y,
                    next_vid, nv->x, nv->y,
                    e->weight, e->angle_leaving_src, e->angle_entering_dst);
            }
        }
    }

    /* Print obstacles */
    fprintf(fp, "\n\n====OBSTACLES====\n");
    for (int i = 0; i < g->num_obstacles; i++) {
        fprintf(fp, "==== OBSTACLE %d =====\n", i + 1);
        for (int j = 0; j < g->obstacles[i].num_points; j++) {
            fprintf(fp, "Point %d: (%.2f, %.2f)\n", j + 1,
                g->obstacles[i].px[j], g->obstacles[i].py[j]);
        }
        fprintf(fp, "\n");
    }

    fclose(fp);
    free(path_nodes);
    free(is_path_vertex);
    printf("Output written to %s\n", filename);
}

```

```

void print_path_validity(Graph *g, Stack *path, double theta_max, double l_min) {
    if (!path || stack_is_empty(path)) {
        printf("No path to validate.\n");
        return;
    }

    int path_len = stack_size(path);
    unsigned int *nodes = (unsigned int *)malloc(sizeof(unsigned int) * path_len);

    /* Extract path without consuming stack */
    Stack *temp = stack_create(path_len);
    for (int i = 0; i < path_len; i++) {
        nodes[i] = stack_pop(path);
        stack_push(temp, nodes[i]);
    }
    for (int i = 0; i < path_len; i++) {
        stack_push(path, stack_pop(temp));
    }
    stack_free(temp);

    double total_length = 0.0;
    int all_valid = 1;

    printf("\n=== PATH VALIDITY REPORT ===\n");
    printf("  theta_max = %.1f deg, l_min = %.1f m (l_min/2 = %.1f m)\n", theta_max, l_min,
l_min / 2.0);
    printf("  Path has %d vertices, %d edges\n\n", path_len, path_len - 1);

    printf("  %-6s  %-20s  %-12s  %-12s  %-8s\n",
        "Edge#", "Segment", "Length(m)", "Turn(deg)", "Valid?");
    printf("  %-6s  %-20s  %-12s  %-12s  %-8s\n",
        "-----", "-----", "-----", "-----", "-----");

    double prev_arrival_angle = -1.0;

    for (int i = 0; i < path_len - 1; i++) {
        Edge *e = find_edge(g, nodes[i], nodes[i + 1]);
        if (!e) {
            printf("  ERROR: No edge from %u to %u!\n", nodes[i], nodes[i + 1]);
            all_valid = 0;
            continue;
        }

        double seg_len = e->weight;
        total_length += seg_len;

        /* Check segment length */
        int len_ok = (seg_len >= l_min / 2.0 - 0.001);

        /* Check turn angle */
        double t = -1.0;
        int turn_ok = 1;
        if (i > 0 && prev_arrival_angle >= 0.0) {
            double continuation = fmod(prev_arrival_angle + 180.0, 360.0);
            double diff = fabs(e->angle_leaving_src - continuation);
            if (diff > 180.0) diff = 360.0 - diff;
            t = diff;
            if (t > theta_max + 0.001) turn_ok = 0;
        }

        int valid = len_ok && turn_ok;
        if (!valid) all_valid = 0;

        char seg_str[32];

```

```

    snprintf(seg_str, sizeof(seg_str), "%u -> %u", nodes[i], nodes[i + 1]);

    if (t >= 0.0) {
        printf(" %6d %20s %12.2f %12.2f %8s%s\n",
            i + 1, seg_str, seg_len, t,
            valid ? "OK" : "FAIL",
            !len_ok ? " [SHORT]" : "",
            !turn_ok ? " [TURN]" : "");
    } else {
        printf(" %6d %20s %12.2f %12.2f %8s%s\n",
            i + 1, seg_str, seg_len, "N/A",
            len_ok ? "OK" : "FAIL",
            !len_ok ? " [SHORT]" : "");
    }

    prev_arrival_angle = e->angle_entering_dst;
}

printf("\n Total path length: %.2f m\n", total_length);
printf(" Overall validity: %s\n\n", all_valid ? "ALL VALID" : "VIOLATIONS FOUND");

free(nodes);
}

/* Silent validation: returns 1 if path is fully valid, 0 otherwise */
int validate_path_silent(Graph *g, Stack *path, double theta_max, double l_min) {
    if (!path || stack_is_empty(path)) return 0;

    int path_len = stack_size(path);
    if (path_len < 2) return 1; /* Trivial path */

    unsigned int *nodes = (unsigned int *)malloc(sizeof(unsigned int) * path_len);
    Stack *temp = stack_create(path_len);
    for (int i = 0; i < path_len; i++) {
        nodes[i] = stack_pop(path);
        stack_push(temp, nodes[i]);
    }
    for (int i = 0; i < path_len; i++) {
        stack_push(path, stack_pop(temp));
    }
    stack_free(temp);

    double prev_arrival_angle = -1.0;
    int all_valid = 1;

    for (int i = 0; i < path_len - 1; i++) {
        Edge *e = find_edge(g, nodes[i], nodes[i + 1]);
        if (!e) { all_valid = 0; break; }

        /* Length check */
        if (e->weight < l_min / 2.0 - 0.001) { all_valid = 0; break; }

        /* Turn check (skip for first edge) */
        if (i > 0 && prev_arrival_angle >= 0.0) {
            double continuation = fmod(prev_arrival_angle + 180.0, 360.0);
            double diff = fabs(e->angle_leaving_src - continuation);
            if (diff > 180.0) diff = 360.0 - diff;
            if (diff > theta_max + 0.001) { all_valid = 0; break; }
        }

        prev_arrival_angle = e->angle_entering_dst;
    }

    free(nodes);
    return all_valid;
}

```

```
}
```

new_tests.h — Additional test prototypes

```
#ifndef NEW_TESTS_H
#define NEW_TESTS_H

/* Test 6: Verify PST query correctness against brute-force */
void run_pst_correctness_test(void);

/* Test 7: Randomized stress testing with automated validation */
void run_random_stress_test(void);

/* Test 8: Edge cases (src==dst, unreachable, theta=0, L_min too large) */
void run_edge_cases_test(void);

/* Test 9: Parameter sweep on theta_max */
void run_parameter_sweep_test(void);

#endif
```

new_tests.c — Tests 6-9: PST correctness, stress, edge cases, parameter sweep

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include "graph.h"
#include "stack.h"
#include "shortest_path.h"
#include "print_output.h"
#include "pst.h"
#include "new_tests.h"

/* Helper to create obstacle from 4 points */
static Obstacle make_obs(double x1, double y1, double x2, double y2,
                        double x3, double y3, double x4, double y4) {
    Obstacle o;
    o.px[0]=x1; o.py[0]=y1;
    o.px[1]=x2; o.py[1]=y2;
    o.px[2]=x3; o.py[2]=y3;
    o.px[3]=x4; o.py[3]=y4;
    o.num_points = 4;
    return o;
}

/* =====
TEST 6: PST Query Correctness - compare against brute-force
===== */

/* Brute-force angle+length filter for validation */
static int brute_force_query(Graph *g, unsigned int vertex, double lo, double hi,
                        double min_len, Edge ***results) {
    int count = 0;
    int cap = 64;
    Edge **res = (Edge **)malloc(sizeof(Edge *) * cap);

    Edge *e = g->adj_list[vertex];
    while (e) {
        double a = e->angle_leaving_src;
        int in_range;
```

```

        if (lo <= hi) {
            in_range = (a >= lo - 0.00001 && a <= hi + 0.00001);
        } else {
            /* Wraparound */
            in_range = (a >= lo - 0.00001 || a <= hi + 0.00001);
        }
        if (in_range && e->weight >= min_len - 0.00001) {
            if (count >= cap) {
                cap *= 2;
                res = (Edge **)realloc(res, sizeof(Edge *) * cap);
            }
            res[count++] = e;
        }
        e = e->next;
    }
    *results = res;
    return count;
}

/* Compare two result sets (unordered) */
static int edge_sets_equal(Edge **a, int na, Edge **b, int nb) {
    if (na != nb) return 0;
    int *matched = (int *)calloc(nb, sizeof(int));
    for (int i = 0; i < na; i++) {
        int found = 0;
        for (int j = 0; j < nb; j++) {
            if (!matched[j] && a[i] == b[j]) {
                matched[j] = 1;
                found = 1;
                break;
            }
        }
        if (!found) { free(matched); return 0; }
    }
    free(matched);
    return 1;
}

void run_pst_correctness_test(void) {
    printf("\n===== \n");
    printf("TEST 6: PST Query Correctness vs Brute-Force \n");
    printf("===== \n");

    /* Build a test graph - moderate size */
    double length = 20.0, height = 20.0, resolution = 1.0;
    double max_edge_dist = 8.0;

    Graph *g = graph_create(length, height, resolution, max_edge_dist, NULL, 0);

    /* Sample vertices: every 5th one, plus edge cases */
    unsigned int samples[20];
    int num_samples = 0;
    for (unsigned int i = 0; i < g->vertex_count && num_samples < 20; i += g->vertex_count /
20) {
        samples[num_samples++] = i;
    }

    /* Query ranges to test */
    double test_queries[][3] = {
        /* lo, hi, min_len */
        {0.0, 90.0, 0.0}, /* basic range, no length filter */
        {0.0, 90.0, 3.0}, /* range + length */
        {90.0, 180.0, 0.0}, /* different quadrant */
        {180.0, 270.0, 2.0},
        {270.0, 360.0, 4.0},

```

```

        {315.0, 45.0, 0.0},      /* wraparound, no length */
        {315.0, 45.0, 3.0},      /* wraparound with length */
        {350.0, 10.0, 2.0},      /* narrow wraparound */
        {45.0, 135.0, 5.0},      /* high length threshold */
        {0.0, 360.0, 0.0},      /* everything */
        {0.0, 360.0, 6.0},      /* everything, high length */
        {10.0, 20.0, 0.0},      /* narrow window */
    };

    int num_queries = 12;

    int total_tests = 0;
    int passed_tests = 0;
    int total_pst_results = 0;
    int total_bf_results = 0;

    for (int s = 0; s < num_samples; s++) {
        unsigned int v = samples[s];
        int edge_count = 0;
        Edge **edge_arr = get_edges_array(g, v, &edge_count);
        if (edge_count == 0) {
            free(edge_arr);
            continue;
        }

        PST *pst = pst_build(edge_arr, edge_count);
        free(edge_arr);

        for (int q = 0; q < num_queries; q++) {
            double lo = test_queries[q][0];
            double hi = test_queries[q][1];
            double min_len = test_queries[q][2];

            Edge **pst_results = NULL;
            int pst_count = pst_query(pst, lo, hi, min_len, &pst_results);

            Edge **bf_results = NULL;
            int bf_count = brute_force_query(g, v, lo, hi, min_len, &bf_results);

            total_tests++;
            total_pst_results += pst_count;
            total_bf_results += bf_count;

            if (edge_sets_equal(pst_results, pst_count, bf_results, bf_count)) {
                passed_tests++;
            } else {
                printf(" MISMATCH: vertex %u, query [%.1f,%.1f] min_len=%.1f: "
                       "PST=%d, BF=%d\n", v, lo, hi, min_len, pst_count, bf_count);
            }

            free(pst_results);
            free(bf_results);
        }

        pst_free(pst);
    }

    printf(" Total queries tested: %d\n", total_tests);
    printf(" Passed: %d / %d (%.1f%%)\n", passed_tests, total_tests,
           100.0 * passed_tests / total_tests);
    printf(" Total PST results: %d, BF results: %d\n",
           total_pst_results, total_bf_results);
    printf(" Result: %s\n", passed_tests == total_tests ? "ALL PASS" : "FAILURES");

    graph_free(g);
}

```

```

/* =====
TEST 7: Randomized Stress Testing
===== */

/* Random double in [a, b) */
static double rand_double(double a, double b) {
    return a + (b - a) * ((double)rand() / RAND_MAX);
}

/* Generate a random obstacle inside the given grid bounds */
static Obstacle random_obstacle(double grid_w, double grid_h, double min_size, double
max_size) {
    double w = rand_double(min_size, max_size);
    double h = rand_double(min_size, max_size);
    double x = rand_double(2.0, grid_w - w - 2.0);
    double y = rand_double(2.0, grid_h - h - 2.0);
    return make_obs(x, y, x, y + h, x + w, y + h, x + w, y);
}

void run_random_stress_test(void) {
    printf("\n===== \n");
    printf("TEST 7: Randomized Stress Testing \n");
    printf("===== \n");

    srand(42); /* Reproducible seed */

    int num_trials = 30;
    int paths_found = 0;
    int paths_valid = 0;
    int paths_no_solution = 0;

    double length = 20.0, height = 20.0, resolution = 1.0;
    double theta_max = 45.0, l_min = 6.0;
    double max_edge_dist = 8.0;

    printf(" Running %d random graph trials... \n", num_trials);
    printf(" Parameters: %dx%d, resolution=%.1f, theta_max=%.0f, l_min=%.1f \n",
        (int)length, (int)height, resolution, theta_max, l_min);

    for (int t = 0; t < num_trials; t++) {
        /* Random number of obstacles (1-4) */
        int num_obs = 1 + (rand() % 4);
        Obstacle *obstacles = (Obstacle *)malloc(sizeof(Obstacle) * num_obs);
        for (int i = 0; i < num_obs; i++) {
            obstacles[i] = random_obstacle(length, height, 2.0, 5.0);
        }

        Graph *g = graph_create(length, height, resolution, max_edge_dist,
            obstacles, num_obs);

        /* Random src and dst */
        int src, dst;
        int attempts = 0;
        do {
            double sx = rand_double(1.0, length - 1.0);
            double sy = rand_double(1.0, height - 1.0);
            double dx = rand_double(1.0, length - 1.0);
            double dy = rand_double(1.0, height - 1.0);
            src = coord_to_id(g, sx, sy);
            dst = coord_to_id(g, dx, dy);
            attempts++;
        } while ((src < 0 || dst < 0 || src == dst ||
            g->adj_count[src] == 0 || g->adj_count[dst] == 0)
            && attempts < 20);
    }
}

```

```

        if (src >= 0 && dst >= 0 && src != dst) {
            Stack *path = shortest_path(g, (unsigned int)src, (unsigned int)dst,
                                       l_min, theta_max);

            if (path) {
                paths_found++;
                if (validate_path_silent(g, path, theta_max, l_min)) {
                    paths_valid++;
                } else {
                    printf(" Trial %d: path found but INVALID (src=%d, dst=%d,
obstacles=%d)\n",
                           t + 1, src, dst, num_obs);
                }
                stack_free(path);
            } else {
                paths_no_solution++;
            }
        }

        graph_free(g);
        free(obstacles);
    }

    printf("\n Results over %d trials:\n", num_trials);
    printf(" Paths found:      %d\n", paths_found);
    printf(" Paths valid:      %d\n", paths_valid);
    printf(" No path exists:    %d\n", paths_no_solution);
    printf(" Validity rate:     %.1f%% (%d/%d)\n",
           paths_found > 0 ? 100.0 * paths_valid / paths_found : 0.0,
           paths_valid, paths_found);
    printf(" Result: %s\n",
           paths_valid == paths_found ? "ALL PATHS VALID" : "VIOLATIONS FOUND");
}

/* =====
TEST 8: Negative / Edge Cases
===== */

void run_edge_cases_test(void) {
    printf("\n===== \n");
    printf("TEST 8: Negative / Edge Cases\n");
    printf("===== \n");

    /* --- Case 1: src == dst --- */
    printf("\n--- Case 1: src == dst ---\n");
    {
        Graph *g = graph_create(10.0, 10.0, 1.0, 5.0, NULL, 0);
        int v = coord_to_id(g, 5.0, 5.0);
        Stack *path = shortest_path(g, (unsigned int)v, (unsigned int)v, 4.0, 45.0);
        if (path) {
            printf(" Path returned with %d vertex(es): ", stack_size(path));
            if (stack_size(path) == 1) printf("TRIVIAL PATH OK\n");
            else printf("UNEXPECTED LENGTH\n");
            stack_free(path);
        } else {
            printf(" NO PATH returned\n");
        }
        graph_free(g);
    }

    /* --- Case 2: Unreachable destination (obstacle wall across) --- */
    printf("\n--- Case 2: Unreachable Destination ---\n");
    {
        /* Wall from top to bottom blocks all paths */
        Obstacle wall[1];

```

```

wall[0] = make_obs(5.0, 0.0, 5.0, 10.0, 6.0, 10.0, 6.0, 0.0);
Graph *g = graph_create(10.0, 10.0, 1.0, 3.0, wall, 1);

int src = coord_to_id(g, 2.0, 5.0);
int dst = coord_to_id(g, 8.0, 5.0);
printf(" src=%d, dst=%d (wall in between, max_edge=3.0m)\n", src, dst);
Stack *path = shortest_path(g, (unsigned int)src, (unsigned int)dst, 4.0, 45.0);
if (path) {
    printf(" Path found (length=%d) - should have been NO PATH\n",
           stack_size(path));
    stack_free(path);
} else {
    printf(" Correctly returned NO PATH\n");
}
graph_free(g);
}

/* --- Case 3: theta_max = 0 (no turns allowed) --- */
printf("\n--- Case 3: theta_max = 0 (straight line only) ---\n");
{
    Graph *g = graph_create(20.0, 20.0, 1.0, 10.0, NULL, 0);
    int src = coord_to_id(g, 2.0, 10.0);
    int dst = coord_to_id(g, 18.0, 10.0);
    printf(" Straight-line path possible: ");
    Stack *path = shortest_path(g, (unsigned int)src, (unsigned int)dst, 4.0, 0.0);
    if (path) {
        printf("FOUND length=%d ", stack_size(path));
        /* With theta=0, only colinear edges allowed - verify */
        if (validate_path_silent(g, path, 0.0, 4.0)) printf("(valid)\n");
        else printf("(invalid!)\n");
        stack_free(path);
    } else {
        printf("NO PATH\n");
    }
    graph_free(g);
}

/* --- Case 4: L_min too large (no edge satisfies length) --- */
printf("\n--- Case 4: L_min larger than all edges ---\n");
{
    Graph *g = graph_create(10.0, 10.0, 1.0, 3.0, NULL, 0);
    int src = coord_to_id(g, 2.0, 5.0);
    int dst = coord_to_id(g, 8.0, 5.0);
    /* L_min/2 = 100, but max edge = 3m, so nothing qualifies */
    printf(" L_min=200 (L_min/2=100, max edge=3m): ");
    Stack *path = shortest_path(g, (unsigned int)src, (unsigned int)dst, 200.0, 45.0);
    if (path) {
        printf("FOUND length=%d (unexpected - all edges too short)\n",
               stack_size(path));
        stack_free(path);
    } else {
        printf("Correctly returned NO PATH\n");
    }
    graph_free(g);
}
}

/* =====
TEST 9: Parameter Sweep on theta_max
===== */

void run_parameter_sweep_test(void) {
    printf("\n===== \n");
    printf("TEST 9: Parameter Sweep on theta_max\n");
    printf("===== \n");
}

```

```

/* Use a fixed obstacle scenario */
double length = 20.0, height = 20.0, resolution = 2.0;
double l_min = 6.0;
double max_edge_dist = 8.0;

/* Obstacle forces the path to make a turn */
Obstacle obstacles[1];
obstacles[0] = make_obs(8.0, 0.0, 8.0, 12.0, 12.0, 12.0, 12.0, 0.0);

double thetas[] = {15.0, 30.0, 45.0, 60.0, 90.0, 180.0};
int num_thetas = 6;

printf("\n Graph: %dx%dm, resolution=%.1f, L_min=%.1f\n",
       (int)length, (int)height, resolution, l_min);
printf(" Obstacle: wall from (8,0) to (12,12)\n");
printf(" Fixed src=(2, 10), dst=(18, 10)\n\n");

printf(" %12s %12s %8s %8s %10s\n",
       "theta_max", "path_cost", "edges", "valid?", "expansions");
printf(" %12s %12s %8s %8s %10s\n",
       "-----", "-----", "-----", "-----", "-----");

for (int i = 0; i < num_thetas; i++) {
    double theta = thetas[i];
    Graph *g = graph_create(length, height, resolution, max_edge_dist,
                           obstacles, 1);
    int src = coord_to_id(g, 2.0, 10.0);
    int dst = coord_to_id(g, 18.0, 10.0);

    if (src < 0 || dst < 0) { graph_free(g); continue; }

    /* Suppress the PST expansions print */
    fflush(stdout);
    int stdout_copy = dup(1);
    FILE *nullf = fopen("/dev/null", "w");
    dup2(fileno(nullf), 1);

    Stack *path = shortest_path(g, (unsigned int)src, (unsigned int)dst,
                               l_min, theta);

    fflush(stdout);
    dup2(stdout_copy, 1);
    fclose(nullf);
    close(stdout_copy);

    /* Compute path cost */
    double cost = 0.0;
    int edge_count = 0;
    int is_valid = 0;
    if (path) {
        int len = stack_size(path);
        edge_count = len - 1;
        unsigned int *nodes = (unsigned int *)malloc(sizeof(unsigned int) * len);
        Stack *temp = stack_create(len);
        for (int j = 0; j < len; j++) {
            nodes[j] = stack_pop(path);
            stack_push(temp, nodes[j]);
        }
        for (int j = 0; j < len; j++) stack_push(path, stack_pop(temp));
        stack_free(temp);

        for (int j = 0; j < len - 1; j++) {
            Edge *e = g->adj_list[nodes[j]];
            while (e) {

```

```

        if (e->dst == nodes[j + 1]) {
            cost += e->weight;
            break;
        }
        e = e->next;
    }
}
free(nodes);
is_valid = validate_path_silent(g, path, theta, l_min);
}

if (path) {
    printf(" %12.1f %12.2f %8d %8s %10s\n",
           theta, cost, edge_count, is_valid ? "YES" : "NO", "-");
    stack_free(path);
} else {
    printf(" %12.1f %12s %8s %8s %10s\n",
           theta, "NO PATH", "-", "-", "-");
}

graph_free(g);
}

printf("\n Observation: As theta_max increases, path cost decreases (more "
       "routing flexibility).\n Very tight constraints may yield NO PATH.\n");
}

```

main.c — Test driver - runs all 9 tests

```

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <string.h>
#include "graph.h"
#include "stack.h"
#include "shortest_path.h"
#include "print_output.h"
#include "new_tests.h"

/* Helper to create an obstacle from 4 corner points */
static Obstacle make_obstacle(double x1, double y1, double x2, double y2,
                              double x3, double y3, double x4, double y4) {
    Obstacle obs;
    obs.px[0] = x1; obs.py[0] = y1;
    obs.px[1] = x2; obs.py[1] = y2;
    obs.px[2] = x3; obs.py[2] = y3;
    obs.px[3] = x4; obs.py[3] = y4;
    obs.num_points = 4;
    return obs;
}

/* =====
TEST 1: Benchmark from Example1_Setup.txt (Double-Channel)
===== */
static void run_benchmark_test(void) {
    printf("\n=====\\n");
    printf("TEST 1: Double-Channel Benchmark\\n");
    printf("=====\\n");

    /* Parameters from Example1_Setup.txt */
    double length = 30.0, height = 30.0;
    double resolution = 2.0; /* 60x60 grid */
    double max_edge_dist = 12.0; /* Needs to be large enough for smooth arcs */
    double theta_max = 45.0;

```

```

double l_min = 8.0;

/* Start: (2.0, 15.0), End: (28.0, 15.0) */
double start_x = 2.0, start_y = 15.0;
double end_x = 28.0, end_y = 15.0;

/* 4 obstacles */
Obstacle obstacles[4];

/* 1. Bottom Block */
obstacles[0] = make_obstacle(10.0, 0.0, 10.0, 13.0, 20.0, 13.0, 20.0, 0.0);
/* 2. Top Block */
obstacles[1] = make_obstacle(10.0, 17.0, 10.0, 26.0, 20.0, 26.0, 20.0, 17.0);
/* 3. Trap Tooth A (Bottom-Middle) */
obstacles[2] = make_obstacle(14.0, 13.0, 14.0, 15.5, 15.0, 15.5, 15.0, 13.0);
/* 4. Trap Tooth B (Top-Middle) */
obstacles[3] = make_obstacle(16.0, 14.5, 16.0, 17.0, 17.0, 17.0, 17.0, 14.5);

Graph *g = graph_create(length, height, resolution, max_edge_dist, obstacles, 4);

int src = coord_to_id(g, start_x, start_y);
int dst = coord_to_id(g, end_x, end_y);
printf("Source vertex: %d (%.1f, %.1f)\n", src, start_x, start_y);
printf("Dest vertex:   %d (%.1f, %.1f)\n", dst, end_x, end_y);

if (src < 0 || dst < 0) {
    printf("ERROR: Invalid source/destination coordinates!\n");
    graph_free(g);
    return;
}

/* Run standard Dijkstra */
printf("\n--- Standard Dijkstra ---\n");
Stack *path_dijkstra = dijkstra_standard(g, (unsigned int)src, (unsigned int)dst);
if (path_dijkstra) {
    print_output(g, path_dijkstra, "bench_dijkstra.txt");
    print_path_validity(g, path_dijkstra, theta_max, l_min);
    stack_free(path_dijkstra);
}

/* Run PST-constrained */
printf("\n--- PST Constrained ---\n");
Stack *path_pst = shortest_path(g, (unsigned int)src, (unsigned int)dst, l_min,
theta_max);
if (path_pst) {
    print_output(g, path_pst, "bench_pst.txt");
    print_path_validity(g, path_pst, theta_max, l_min);
    stack_free(path_pst);
}

graph_free(g);
}

/* =====
TEST 2: Open field - no obstacles (all algorithms should agree)
===== */
static void run_open_field_test(void) {
    printf("\n===== \n");
    printf("TEST 2: Open Field (No Obstacles)\n");
    printf("===== \n");

    double length = 20.0, height = 20.0;
    double resolution = 1.0;
    double max_edge_dist = 10.0;
    double theta_max = 45.0;

```

```

double l_min = 4.0;

Graph *g = graph_create(length, height, resolution, max_edge_dist, NULL, 0);

int src = coord_to_id(g, 2.0, 10.0);
int dst = coord_to_id(g, 18.0, 10.0);
printf("Source: %d, Dest: %d\n", src, dst);

if (src < 0 || dst < 0) { graph_free(g); return; }

printf("\n--- Standard Dijkstra ---\n");
Stack *p1 = dijkstra_standard(g, (unsigned int)src, (unsigned int)dst);
if (p1) {
    print_path_validity(g, p1, theta_max, l_min);
    stack_free(p1);
}

printf("\n--- PST Constrained ---\n");
Stack *p2 = shortest_path(g, (unsigned int)src, (unsigned int)dst, l_min, theta_max);
if (p2) {
    print_path_validity(g, p2, theta_max, l_min);
    stack_free(p2);
}

graph_free(g);
}

/* =====
TEST 3: U-Turn scenario - wall forces a detour
===== */
static void run_urn_test(void) {
    printf("\n===== \n");
    printf("TEST 3: U-Turn Scenario\n");
    printf("===== \n");

    double length = 20.0, height = 20.0;
    double resolution = 2.0;
    double max_edge_dist = 8.0;
    double theta_max = 30.0; /* Tight turn constraint */
    double l_min = 6.0;

    /* Wall across the middle with a gap at the top */
    Obstacle obstacles[1];
    obstacles[0] = make_obstacle(9.0, 0.0, 9.0, 15.0, 11.0, 15.0, 11.0, 0.0);

    Graph *g = graph_create(length, height, resolution, max_edge_dist, obstacles, 1);

    int src = coord_to_id(g, 5.0, 5.0);
    int dst = coord_to_id(g, 15.0, 5.0);
    printf("Source: %d, Dest: %d\n", src, dst);

    if (src < 0 || dst < 0) { graph_free(g); return; }

    printf("\n--- Standard Dijkstra ---\n");
    Stack *p1 = dijkstra_standard(g, (unsigned int)src, (unsigned int)dst);
    if (p1) {
        print_output(g, p1, "uturn_dijkstra.txt");
        print_path_validity(g, p1, theta_max, l_min);
        stack_free(p1);
    }

    printf("\n--- PST Constrained ---\n");
    Stack *p2 = shortest_path(g, (unsigned int)src, (unsigned int)dst, l_min, theta_max);
    if (p2) {
        print_output(g, p2, "uturn_pst.txt");
    }
}

```

```

        print_path_validity(g, p2, theta_max, l_min);
        stack_free(p2);
    }

    graph_free(g);
}

/* =====
TEST 4: Resolution scaling - same scenario at different resolutions
===== */
static void run_resolution_test(void) {
    printf("\n=====\\n");
    printf("TEST 4: Resolution Scaling\\n");
    printf("=====\\n");

    double length = 20.0, height = 20.0;
    double theta_max = 45.0;
    double l_min = 6.0;
    double max_edge_dist = 10.0;

    /* Simple corridor with a bend */
    Obstacle obstacles[2];
    obstacles[0] = make_obstacle(8.0, 0.0, 8.0, 8.0, 12.0, 8.0, 12.0, 0.0);
    obstacles[1] = make_obstacle(8.0, 12.0, 8.0, 20.0, 12.0, 20.0, 12.0, 12.0);

    double resolutions[] = {1.0, 2.0};
    int num_res = 2;

    for (int ri = 0; ri < num_res; ri++) {
        double res = resolutions[ri];
        printf("\\n--- Resolution: %.1f cells/m ---\\n", res);

        Graph *g = graph_create(length, height, res, max_edge_dist, obstacles, 2);
        int src = coord_to_id(g, 2.0, 10.0);
        int dst = coord_to_id(g, 18.0, 10.0);

        if (src < 0 || dst < 0) { graph_free(g); continue; }

        printf("  Standard Dijkstra:\\n");
        Stack *p1 = dijkstra_standard(g, (unsigned int)src, (unsigned int)dst);
        if (p1) {
            print_path_validity(g, p1, theta_max, l_min);
            stack_free(p1);
        }

        printf("  PST Constrained:\\n");
        Stack *p2 = shortest_path(g, (unsigned int)src, (unsigned int)dst, l_min, theta_max);
        if (p2) {
            char fname[64];
            snprintf(fname, sizeof(fname), "resolution_%.0f_pst.txt", res);
            print_output(g, p2, fname);
            print_path_validity(g, p2, theta_max, l_min);
            stack_free(p2);
        }

        graph_free(g);
    }
}

/* =====
TEST 5: Zigzag corridor (micro-maneuver trap)
===== */
static void run_zigzag_test(void) {
    printf("\n=====\\n");
    printf("TEST 5: Zigzag Corridor (Micro-maneuver Trap)\\n");

```

```

printf("=====\n");

double length = 30.0, height = 20.0;
double resolution = 2.0;
double max_edge_dist = 10.0;
double theta_max = 45.0;
double l_min = 8.0;

/* Create alternating teeth to force a zigzag */
Obstacle obstacles[4];
obstacles[0] = make_obstacle(7.0, 0.0, 7.0, 8.0, 9.0, 8.0, 9.0, 0.0);
obstacles[1] = make_obstacle(7.0, 12.0, 7.0, 20.0, 9.0, 20.0, 9.0, 12.0);
obstacles[2] = make_obstacle(14.0, 0.0, 14.0, 10.0, 16.0, 10.0, 16.0, 0.0);
obstacles[3] = make_obstacle(14.0, 14.0, 14.0, 20.0, 16.0, 20.0, 16.0, 14.0);

Graph *g = graph_create(length, height, resolution, max_edge_dist, obstacles, 4);

int src = coord_to_id(g, 2.0, 10.0);
int dst = coord_to_id(g, 28.0, 10.0);
printf("Source: %d, Dest: %d\n", src, dst);

if (src < 0 || dst < 0) { graph_free(g); return; }

printf("\n--- Standard Dijkstra ---\n");
Stack *p1 = dijkstra_standard(g, (unsigned int)src, (unsigned int)dst);
if (p1) {
    print_output(g, p1, "zigzag_dijkstra.txt");
    print_path_validity(g, p1, theta_max, l_min);
    stack_free(p1);
}

printf("\n--- PST Constrained ---\n");
Stack *p2 = shortest_path(g, (unsigned int)src, (unsigned int)dst, l_min, theta_max);
if (p2) {
    print_output(g, p2, "zigzag_pst.txt");
    print_path_validity(g, p2, theta_max, l_min);
    stack_free(p2);
}

graph_free(g);
}

/* ===== */
int main(void) {
    printf("CS 3050 A4 - Constrained Shortest Paths\n");
    printf("Tianyi Zhang\n\n");

    run_benchmark_test();
    run_open_field_test();
    run_urn_test();
    run_resolution_test();
    run_zigzag_test();

    /* Additional rigor tests */
    run_pst_correctness_test();
    run_edge_cases_test();
    run_parameter_sweep_test();
    run_random_stress_test();

    printf("\n=====\n");
    printf("All tests complete.\n");
    printf("=====\n");
    return 0;
}

```

